ADDIS ABABA UNIVERSITY
FACULTY OF NATURAL AND COMPUTATIONAL SCIENCES

DEPARTMENT OF COMPUTER SCIENCE

KULLI: P2P Logistics Mobile Application On Demand Trucking Service

FINAL PROJECT I BY:

Bereket Berehanu	NSR/6732/14
Chris Tedla	UGR/2624/15
Dawit TekleBirhan	UGR/0426/15
Lamesgnew Awdro	NSR/4291/14

PROJECT ADVISOR: **Dr. Amsale Z.**

Jan, 2026

iii. DECLARATION

This is to declare that this project work which is done under the supervision of **Dr. Amsale Z.** and having the title **KULLI: P2P Logistics Mobile Application On Demand Trucking Service** is the sole contribution of:

Bereket Berehanu
Chris Tedla
Dawit Teklebirhan
Lamesgnew Awdro

No part of the project work has been reproduced illegally (copy and paste) which can be considered as Plagiarism. All referenced parts have been used to argue the idea and have been cited properly. We will be responsible and liable for any consequence if violation of this declaration is proven.

Date: __January 28, 2026__

Group Members:

Bereket Berehanu
Chris Tedla
Dawit Teklebirhan
Lamesgnew Awdro

Full Name	**Signature**
_____Bereket Berehanu_____	_____Bereket_____

iv. CERTIFICATE

I certify that this BSc **Final year project** deliverable entitled **KULLI: P2P Logistics Mobile Application On Demand Trucking Service** by:

Bereket Berehanu
Chris Tedla
Dawit Teklebirhan
Lamesgnew Awdro

is approved by me for submission. I certify further that, to the best of my knowledge, the report represents work carried out by the students.

Jan 28, 2026	Dr. Amsale Z.
Date	**Name and Signature of Supervisor**

ABSTRACT

The Kuli project introduces a peer-to-peer mobile marketplace to resolve the fragmentation and pricing ambiguity currently found in Addis Ababa's informal truck logistics sector. Developed with a React Native frontend and a modular Node.js backend, the system connects clients with verified truck owners using proximity-based matching and transparent cost estimation. It features secure multi-role authentication and a call-assisted booking service to bridge the gap for users with limited digital literacy. Ultimately, the platform fosters trust and economic growth by formalizing service discovery for households and independent logistics providers.

Acknowledgment

I am profoundly grateful to my academic advisor for the invaluable guidance and feedback that directed the development of this project. I also wish to thank the team at Family Movers for providing the operational insights and workflow validation essential to aligning the system with local industry practices. My thanks extend to the independent truck owners and stakeholders who participated in surveys, providing the primary data that grounded our design decisions. Lastly, I appreciate my family and colleagues for their unwavering support during the 32-week roadmap of this project.

TABLE OF CONTENT

1. PROJECT PROPOSAL

1.1. Introduction

Logistics services that require truck-based transportation are a frequent necessity for individuals and small businesses. However, many users continue to face significant challenges in securing reliable, affordable, and transparent trucking services for transporting goods and personal belongings. In most cases, clients depend on informal networks or costly service providers, which often result in stress, unpredictable pricing, limited service availability, and avoidable delays. At the same time, independent truck owners struggle to consistently find customers due to limited visibility, weak marketing channels, and the absence of structured digital platforms that connect them with potential clients.

To address these challenges, this project proposes a peer-to-peer (P2P) mobile application, named **Kuli**, that connects clients requiring truck-based logistics services including household relocation as a primary use case with verified independent truck owners, creating a simple, secure, and transparent digital marketplace that optimizes booking, communication, and service delivery.

1.2 Statement of the Problem and Justification

Clients frequently experience difficulty finding trustworthy and available truck owners, comparing prices and service quality, scheduling transportation services efficiently, and communicating clearly to ensure reliable delivery. Similarly, independent truck owners face challenges such as limited visibility to potential customers, inconsistent job opportunities, and the lack of a centralized platform to manage bookings and service requests. These issues result in unreliable service delivery, unclear pricing, frequent delays, and frustration for both parties.

Therefore, a centralized mobile application is needed to improve trust, efficiency, and accessibility in truck-based logistics services, particularly for household relocation and similar transportation needs.

1.3 Project Objective

1.3.1 General Objective of the System

The general objective of this study is to design a peer-to-peer mobile application that facilitates efficient, reliable, and transparent truck-based logistics services by connecting clients with verified independent truck owners through a centralized digital platform.

1.3.2 Specific Objectives of the System

- To analyze existing truck-based logistics and household relocation processes and identify key challenges related to trust, accessibility, pricing transparency, and communication.
- To design a secure registration and authentication mechanism that verifies both clients and independent truck owners before they access the platform.
- To enable clients to create detailed transportation requests, including pickup location, destination, distance, type and volume of items, preferred service date, and required truck type.
- To design a proximity-based listing mechanism that allows clients to view available truck owners based on geographic closeness to the pickup location, reducing delays and improving service efficiency.
- To allow independent truck owners to browse and accept suitable transportation requests based on availability, vehicle capacity, and location.
- To design a service ranking and filtering system that lists truck owners based on non-price criteria such as user ratings, vehicle type, proximity, and past service performance.
- To incorporate a rating and review system that enables clients to evaluate completed services and promotes accountability and trust within the platform.
- To provide notifications that keep users informed of job updates and service status changes.
- To design an administrative dashboard for user management, operational oversight, and platform quality control.

1.4 Scope of the Project

The proposed platform will initially operate within a single city, specifically Addis Ababa, and will target two primary user groups: clients seeking truck-based logistics services and independent truck owners providing transportation services. Household relocation will serve as the primary use case of the system, while the platform remains flexible enough to support other transportation needs such as small business deliveries and equipment transport.

Core system features will include secure user registration and authentication, service request posting and acceptance, internal messaging for coordination, a rating and review mechanism to enhance trust, basic price estimation, and an administrative dashboard for platform monitoring. Additionally, the system will support a call-assisted booking service for users without smartphones or limited digital literacy, allowing call-center assistants to create and manage requests on their behalf.

The initial version of the platform will have certain limitations. Real-time GPS tracking of trucks will not be implemented in the first phase, and integrated digital payment services may be introduced in later versions. The platform will function solely as a connector between clients and truck owners and will not be responsible for disputes, damages, or physical handling of goods. Advanced pricing optimization and machine-learning-based automation will also be excluded from the first phase.

1.5 System Development Methodology

The study combines primary data collected from system stakeholders with secondary data sourced from existing logistics platforms and academic literature. Basic analytical techniques, including descriptive analysis of survey data and system modeling, will be used to transform collected data into system requirements and design decisions, ensuring that the system addresses real user needs.

1.5.1 Investigation (Fact-Finding) Methods

Information will be gathered from multiple sources to understand user needs and logistics workflows. This includes individual clients, independent truck owners, and workers or truck owners employed at **Family Movers**, a well-known local moving company in Addis Ababa, to gain practical insights into real-world transportation operations and challenges.

1.5.1.1 Stakeholder Identification and Selection

The primary stakeholders for the system are:

- Clients (renters, students, and small businesses)
- Independent Truck Owners
- Call-Center Assistants
- System Administrators

A purposive sampling strategy will target these groups efficiently. Clients will be engaged through digital channels, such as local Telegram housing groups and university networks, to gather requirements from frequent movers. **Family Movers** will serve as a supporting subject matter expert, providing operational insights, pricing information, and workflow validation to align the system with industry practices. Independent Truck Owners will be interviewed at high-density locations, e.g., near furniture shops or electronics stores, to understand requirements for transporting large items.

1.5.1.2 Surveys and Questionnaires

Structured surveys will be conducted both online and offline to gather data on service demand, current logistics practices, challenges faced, preferred booking methods.

1.5.1.3 Secondary Data and Existing Survey Analysis

Secondary data will be collected from academic journals, research reports, and studies of existing logistics platforms such as Uber, Bolt, and other open-source or publicly documented systems (e.g., OpenStreetMap, OpenTripPlanner, Open Logistics datasets) to support and validate system design decisions.

1.5.2 System Development Tools

1.5.2.1 System Architecture

The system will adopt a client–server architecture with a modular monolithic backend, separating core functions such as authentication, job management, matching, messaging, notifications, and administration. This architecture is selected to simplify development and deployment while ensuring clear separation of concerns, making the system easier to understand, maintain, and extend during the project lifecycle.

This architectural style is chosen because it provides a balance between simplicity and scalability. It reduces development and operational complexity compared to microservices while maintaining clear separation of concerns within the system. The modular structure also allows future migration to microservices if system scale and usage increase.

1.5.2.2 Development Tools and Technologies

- Frontend: React Native for cross-platform mobile development
- Backend: Node.js with a structured framework to support modularization
- Database: MongoDB + Geospatial Indexing
- Authentication: Supabase Auth
- UI/UX Design: Figma for interface prototyping
- APIs: RESTful APIs for communication between client and server

1.5.2.3 Testing and Quality Assurance

Testing will include unit, integration, and functional testing using tools such as Jest, Postman, and React Native Testing Library, supported by Git-based version control and code quality tools.

1.6 Significance of the Project

The system improves access to reliable truck-based logistics services by offering verified service providers, transparent pricing, and efficient communication. Features such as proximity-based matching, messaging, ratings, and call-assisted booking enhance trust, inclusivity, and operational efficiency.

The platform also empowers independent truck owners by increasing visibility and enabling consistent job opportunities, contributing to the formalization and growth of small-scale logistics services.

1.7 Beneficiaries

Primary beneficiaries include individuals, households, and small businesses requiring transportation services, particularly for household relocation. The platform provides access to verified truck owners, transparent service options, proximity-based matching, and efficient

communication, which reduces delays, lowers search costs, and minimizes logistics-related stress.

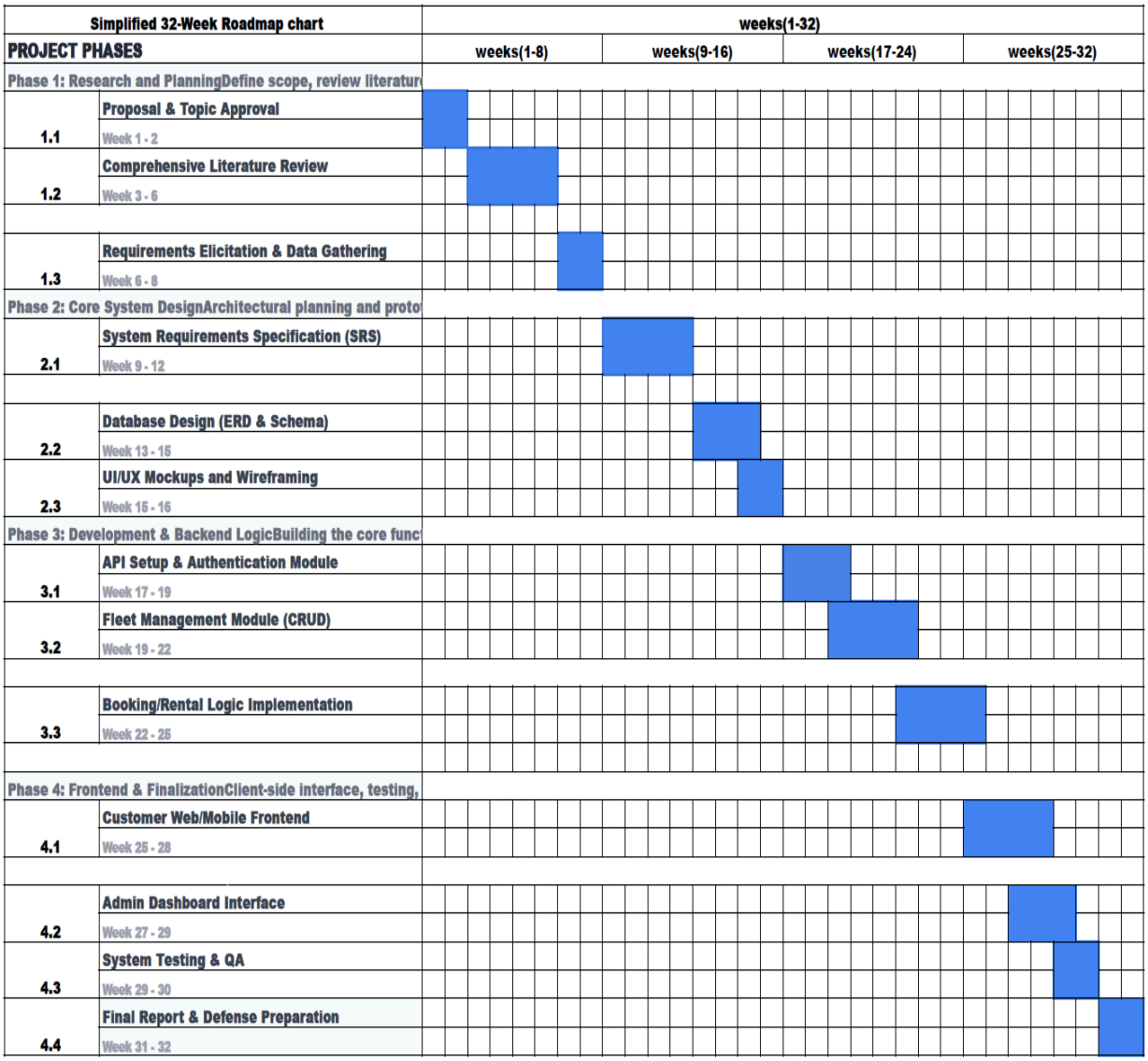
Independent truck owners also benefit from increased visibility, structured and consistent job opportunities, and simplified booking management. The rating and review system helps them build credibility, while proximity-based matching reduces idle time and operational inefficiencies, contributing to more stable income generation.

The product owner (platform operator) benefits through a sustainable revenue model. The platform can generate income by charging a small commission on each successfully completed transaction, service or booking fees, optional premium listings for truck owners, and potential cancellation fees. These revenue streams support system maintenance, operational costs, scalability, and future feature enhancements.

System administrators and support staff benefit from improved tools for user management, service monitoring, and operational oversight, ensuring platform reliability and quality control. Overall, the platform creates economic value for all stakeholders by improving trust, efficiency, and transparency within the local logistics ecosystem.

1.8 Time Schedule

Simplified 32-Week Roadmap



2. REQUIREMENT ANALYSIS

2.1. Introduction

The purpose of the system is to develop a peer-to-peer (P2P) mobile application, Kuli, that facilitates on-demand truck-based logistics services within Addis Ababa. The system serves as a centralized digital marketplace connecting clients who require transportation of goods such as household items, furniture, appliances, and small-scale business deliveries with verified independent truck owners. By enabling proximity-based service discovery, direct communication, transparent service information, and a call-assisted booking option, the system aims to address inefficiencies, trust issues, and pricing ambiguity present in the current logistics market.

2.2. Current System

2.2.1. Major Functions of the Current System

Currently, truck-based logistics services operate through informal and fragmented processes. Clients typically locate truck owners by physically visiting known truck parking areas, relying on word-of-mouth referrals, informal brokers, or personal contacts. Negotiations regarding price, availability, and service details are conducted verbally and often vary significantly depending on negotiation skills, urgency, and perceived demand. Some clients choose established moving companies for reliability; however, these services are generally more expensive and less flexible. These approaches are used not only for household relocation but also for transporting furniture, appliances, and goods for small businesses.

2.2.2. Problems of the Existing System

The existing system presents several challenges:

- **Pricing Inconsistency:** Costs are determined through informal negotiation, leading to unpredictable and sometimes inflated pricing.
- **Difficulty in Service Discovery:** Clients expend considerable time and effort locating available trucks without assurance of success.
- **Lack of Trust and Accountability:** The absence of formal verification exposes clients to risks such as service cancellation, damage, or theft.

- **Operational Inefficiency:** Truck owners experience idle time due to limited visibility, while clients struggle to compare vehicle types and capacities.

2.3. Requirement Gathering

2.3.1. Requirement Gathering Methodologies

A mixed-method approach was used to collect system requirements:

- **Online Surveys:** An online survey was conducted using a purposive sampling strategy to gather requirements from users with prior experience in household relocation and truck-based logistics services. The survey was primarily distributed through local Telegram housing channels and university-related groups, which are commonly used by frequent movers. This approach enabled efficient access to relevant participants and ensured that the collected responses reflected real user challenges and expectations.
- **Informal Interviews and Stakeholder Consultation:** Discussions with independent truck owners and workers or truck owners associated with **Family Movers (**well-known local moving company in Addis Ababa) to understand real operational workflows and constraints.
- **Secondary Data Analysis:** Review of academic studies and publicly available documentation of logistics and ride-hailing platforms (e.g., Uber, Bolt) to benchmark features and best practices.

2.3.2. Results Found

The requirement analysis revealed the following key findings:

- Strong demand for **verified and trustworthy truck owners** to ensure safety and reliability.
- High importance is placed on **accurate and transparent price estimation**, ensuring users receive clear cost estimates based on distance, item type, and truck capacity prior to booking.
- The need for **alternative booking methods** due to varying levels of digital literacy, justifying call-assisted services.

- Preference for **proximity-based matching** to reduce waiting times and transportation delays.

2.4. Proposed System

2.4.1. Overview

The proposed system, **Kuli**, is a mobile-based P2P logistics platform designed to support general truck-based transportation services. It connects clients with verified independent truck owners for multiple logistics use cases, including household relocation, item transportation, and small-scale commercial deliveries. The system supports service request creation, proximity-based matching, communication, notifications, ratings, administrative oversight, and a call-assisted booking mechanism to enhance accessibility.

2.4.2. Functional Requirements

FR-1 User Accounts and Authentication

- **FR-1.1**: The system shall allow users to **register** as one of four roles: **Client**, **Truck Owner**, **Call-Center Assistant**, or **Administrator**.
- **FR-1.2**: The system shall support **email/phone-based registration** and **password** or **OTP** authentication for all roles.
- **FR-1.3**: The system shall support **role-based login** and present role-specific dashboards immediately after authentication.
- **FR-1.4**: The system shall allow users to **reset passwords** and recover accounts via secure OTP or email links.

FR-2 Truck Owner Verification and Onboarding

- **FR-2.1**: The system shall provide a **verification workflow** for truck owners that collects vehicle details, identity documents, and proof of ownership/insurance.
- **FR-2.2**: The system shall allow administrators to **review, approve, or reject** truck owner applications and record the decision with timestamps.
- **FR-2.3**: The system shall prevent unverified truck owners from accepting or appearing in public listings.

- **FR-2.4**: The system shall notify truck owners of verification status changes via in-app notification and SMS/email.

FR-3 Service Request Creation and Management

- **FR-3.1**: The system shall allow clients to **create a logistics service request** specifying: pickup location, destination, item type, item dimensions/volume, weight, preferred pickup date/time, and any special handling instructions.
- **FR-3.2**: The system shall validate required fields and provide inline guidance for acceptable values (e.g., max volume per truck type).
- **FR-3.3:** The system shall allow clients to **cancel** a submitted request within configurable policy windows.

FR-4 Matching, Discovery, and Filtering

- **FR-4.1:** The system shall present a **proximity-based list** of available verified truck owners sorted by distance, availability and aggregate rating..
- **FR-4.2**: The system shall provide filters for **truck type, capacity.**
- **FR-4.3**: The system shall support **automatic matching** that suggests the top N truck owners for a request based on distance, capacity, availability, ratings, and dispute history.

FR-5 Pricing, Quotation, and Payments

- **FR-5.1:** The system shall **calculate and display trip cost estimates** using configurable and market-adjustable pricing rules that consider truck type, load size or volume, travel distance, estimated trip duration, toll charges, prevailing fuel prices, and optional services (e.g., loading assistance), with support for future extension to additional vehicle types.
- **FR-5.2**: The system shall support **multiple payment flows**: pay-on-acceptance, pay-on-delivery, and pay-in-advance (digital payments may be implemented later).
- **FR-5.3**: The system shall record **payment transactions.**

FR-6 Trip Execution and Real-Time Tracking

- **FR-6.1**: The system shall provide **real-time status updates** for each shipment: Requested, Matched, Accepted, En Route to Pickup, Loading, In Transit, Unloading, Completed, Cancelled..
- **FR-6.2:** The system shall allow authorized truck owners or drivers to manually **update predefined shipment statuses** (e.g., Accepted, En Route to Pickup, Loading, Unloading)
- **FR-6.3**: The system shall maintain a **complete event log** for each trip with timestamps.

FR-7 Communication and Notifications

- **FR-7.1**: The system shall provide **in-app messaging** between clients and truck owners with message history tied to the service request.
- **FR-7.2**: The system shall support **call-assisted booking** where authorized call-center assistants can create or modify requests on behalf of clients and record the operator ID.
- **FR-7.3**: The system shall send **notifications** (in-app, SMS, email) for key events: request submission, acceptance, driver en route, arrival, delivery completion, cancellations, and payment receipts.
- **FR-7.4**: The system shall allow users to configure **notification preferences** and opt out of non-essential messages.

FR-8 Ratings, Reviews, and Dispute Resolution

- **FR-8.1:** The system shall allow clients to **submit numeric ratings** and textual reviews for completed trips.
- **FR-8.2:** The system shall **compute an aggregate rating** score for each truck owner based on completed service ratings.
- **FR-8.3:** The system shall make aggregate ratings available to the matching and filtering module to **influence provider ranking and selection**.

- **FR-8.4:** The system shall provide a dispute workflow where clients or truck owners can submit complaints with supporting evidence; administrators can review, mediate, and record outcomes.
- **FR-8.5:** The system shall flag or **penalize truck owners** with unresolved or frequent disputes, affecting their visibility in matching results.

FR-9 Administration and Reporting

- **FR-9.1**: The system shall provide an **administrative dashboard** for user management, service oversight, verification queue, pricing rules, and system health metrics.

FR-10 Truck Owner Dashboard

- **FR-7.1:** The system shall **provide a dashboard** for registered truck owners to manage their activities.
- **FR-7.2:** The system shall allow truck owners to **view and respond to trip requests**, and monitor active and completed trips.
- **FR-7.3:** The system shall allow truck owners to **update trip status** (e.g., Accepted, Loading, In Transit, Completed).
- **FR-7.4:** The system shall allow truck owners to **manage their availability.**
- **FR-7.5:** The system shall allow truck owners to **view their aggregated ratings** .

2.4.3. Non-Functional Requirements

2.4.3.1. Business Rules

- **NFR-1.1 Single Active Request Rule**: A client may have up to **one active pickup request per shipment**; business rules for concurrent requests will be configurable by admin.
- **NFR-1.2 Verification Requirement**: Only **verified truck owners** shall be eligible to receive or accept requests.
- **NFR-1.3 Cancellation Policy**: Cancellation windows, fees, and penalties shall be configurable and enforced by the system.

2.4.3.2. User Interface and Human Factors

- **NFR-2.1 Intuitive Role-Specific UI**: The UI shall be **role-specific**, minimizing visible options to only those relevant to the user's role.
- **NFR-2.2 Mobile-First Design**: The system shall be **mobile-friendly** and usable on standard smartphones with screens down to 4.7 inches.
- **NFR-2.3 Accessibility**: The UI shall meet basic accessibility guidelines (contrast, scalable text, touch target sizes) to support users with limited digital literacy.
- **NFR-2.4 Local Language Support**: The system shall support the primary local language(s) (planned for future implementation) and English for menus and key workflows.

2.4.3.3. Documentation

- **NFR-3.1 User Guides**: Provide **concise user manuals** for Clients, Truck Owners, and Call-Center Assistants with step-by-step workflows and screenshots.
- **NFR-3.2 Technical Documentation**: Provide API documentation, deployment guides, and maintenance procedures for developers and system maintainers.
- **NFR-3.3 Change Log**: Maintain a versioned change log for releases and configuration changes.

2.4.3.4. Hardware Consideration

- **NFR-4.1 Supported Devices**: The mobile application shall support **Android 8+** and **iOS 12+** and degrade gracefully on older devices.
- **NFR-4.2 GPS and Sensors**: The system shall use device GPS for location services and allow manual location entry when GPS is unavailable.
- **NFR-4.3 No Specialized Hardware**: No specialized external hardware shall be required for normal operation.

2.4.3.5. Performance Characteristics

- **NFR-5.1 Response Time**: The system shall respond to user interactions within **3 seconds** under normal load and within **5 seconds** under peak city-wide load.

- **NFR-5.2 Concurrency**: The system shall support concurrent users at a city scale with horizontal scaling options for peak demand.
- **NFR-5.3 Throughput**: The system shall process booking submissions and status updates with a throughput sufficient to handle peak hourly request volumes (to be defined during capacity planning).

2.4.3.6. Error Handling and Extreme Conditions

- **NFR-6.1 Graceful Degradation**: Core features (request creation, status updates, notifications) shall remain available under partial system failures; non-critical features may be temporarily disabled.
- **NFR-6.2 Clear Error Messaging**: The system shall present **user-friendly error messages** with actionable guidance for common issues (invalid input, network loss, authentication failure).
- **NFR-6.3 Retry and Offline Support**: The mobile client shall queue critical actions for retry when connectivity is restored and inform users of offline status.

2.4.3.7. Quality Issues

- **NFR-7.1 Availability Target**: The system shall aim for **99.9% uptime** excluding scheduled maintenance windows.
- **NFR-7.2 Monitoring and Alerts**: Implement monitoring for performance, errors, and security events with automated alerts to administrators.

2.4.3.8. System Modifications

- **NFR-8.1 Modular Architecture**: The system shall be modular to allow incremental feature additions (digital payments, advanced GPS tracking) with minimal downtime.
- **NFR-8.2 CI/CD and Rollback**: Support continuous integration and deployment pipelines with automated tests and safe rollback mechanisms.

2.4.3.9. Physical Environment

- **NFR-9.1 Urban Deployment**: The system shall be optimized for urban environments (Addis Ababa initial deployment) and tolerate variable mobile network quality.
- **NFR-9.2 Data Center and Cloud**: The system shall be deployable to cloud infrastructure with region-aware redundancy.

2.4.3.10. Security Issues

- **NFR-10.1 Authentication and Authorization**: Enforce secure authentication, multi-factor options, and strict role-based access control.
- **NFR-10.2 Data Protection**: Encrypt sensitive data in transit (TLS) and at rest; store minimal personally identifiable information and support data deletion requests.
- **NFR-10.3 Audit Trails**: Maintain tamper-evident audit logs for security-relevant events.

2.4.3.11. Resource Issues

- **NFR-11 .1 Efficient Resource Use**: The system shall minimize bandwidth and storage usage on mobile devices and support incremental data sync.

2.5. Constraints / Pseudo Requirements

The following key real-world constraints represent the major bottlenecks that may affect the system. Each includes an impact description and suggested mitigation:

1. **Limited Truck Availability and Uneven Supply**
   Impact: Peak demand periods may produce long waits and unmet requests, reducing user satisfaction.
   Mitigation: Implement surge indicators, waitlists, driver incentives, and partnerships with fleet operators to smooth supply.

2. **Verification and Onboarding Delays**
   Impact: Manual vetting slows driver onboarding and limits available supply during launch.
   Mitigation: Combine automated document checks with prioritized manual review and provisional statuses for early visibility.

3. **Limited insurance coverage increases financial and legal risk**

   Impact**:** Limited insurance coverage increases financial and legal risk.

   Mitigation**:** Require truck owners to provide vehicle insurance where available, disclaim coverage for transported goods, and allow optional cargo insurance if it becomes available in the future.

4. **Cash-First Economy and Low Digital Payment Adoption**

   Impact: Cash settlements complicate commission collection and dispute resolution.

   Mitigation: Support cash workflows while incentivizing digital payments and implement reconciliation processes.

5. **Network Connectivity and Device Limitations**

   Impact: Intermittent data and older devices reduce app reliability for a portion of users.

   Mitigation: Provide lightweight UI, offline-capable forms for call agents, and SMS/USSD fallbacks for critical notifications.

6. **Geolocation and Address Accuracy**

   Impact: Inaccurate or nonstandard addresses cause misplaced pickups and failed jobs.

   Mitigation: Allow manual pin adjustments, require address confirmation, and use call-assisted verification for ambiguous locations.

7. **Data Quality and Fraud Risk**

   Impact: Fake profiles, manipulated ratings, and inaccurate listings undermine trust and platform credibility.

   Mitigation: Enforce multi-factor verification, periodic audits, and anomaly detection to flag suspicious activity.

8. **Call-Assisted Capacity Limits**

   Impact: Limited call-center throughput constrains service access for non-digital users.

   Mitigation: Scale call team to expected demand, implement templated request flows, and provide supervisors with workload dashboards.

9. **Traffic Congestion and Road Conditions**

   Impact: Urban congestion and poor roads produce unpredictable trip times and higher costs.

   Mitigation: Use conservative ETAs, allow driver route adjustments, and provide congestion warnings to clients.

10. **Fuel Price Volatility and Operating Cost Sensitivity**

    Impact: Rapid fuel cost changes can render fixed estimates unprofitable, increasing

cancellations.

Mitigation: Include adjustable cost components, periodic fare reviews, and optional fuel surcharges during price spikes.

2.6. System Model

2.6.1. Scenario

This section contains the major real life scenarios of the system use cases described in a structured flow of events.

P.S In the current section, '/' is used to convey "one of these" while ',' is used to convey "all of them".

2.6.1-1 Create Account

Table 2.1: Create Account

Scenario ID	**SC-001**
Scenario Name	**Create Account**
Participating Actors	**Client / Truck Owner**
Entry Conditions	Actor has a phone number or an email account
Flow Of Events	1. Actor accesses the application and is directed to the landing page 2. Actor selects the **Create Account** button to enter the **registration** page 3. Actor picks the appropriate **role** from **Client / Truck Driver** 4. Actor will pick between a **phone number**, an **email**

	account, or **both** with one as primary 5. Actor will enter the other required details, including the password 6. System will verify **validity** and **availability** of the entered credentials 7. If successful the system will **create** the account and **login** the actor 8. The system **redirects** to the home page
End Conditions	1. The Account is created and the actor is **logged in** and redirected to the home page 2. The Credentials are invalid and the actor is prompted to **try again**
Quality Requirements	1. The actor will input **valid credentials**

2.6.1-2 Login

Table 2.2: Login

Scenario ID	**SC-002**
Scenario Name	**Login**
Participating Actors	**Client / Truck Owner / Call Center Assistant**
Entry Conditions	Client has been registered and has an account

Flow Of Events	1. Actor accesses the application and selects the **Log In** button 2. Actor **enters** the **credentials**, including the **email/phone number** and **password** 3. System **validates** the account credentials and determines the **account type** 4. If successful, actor is **redirected** to the **appropriate home page**
End Conditions	1. Actor has **logged into** the system 2. Actor has inputted **invalid credentials** and is prompted to **try again**
Quality Requirements	1. Actor **remembers** and **inputs** the **correct credentials**

2.6.1-3 Register Vehicle

Table 2.3: Register Vehicle

Scenario ID	**SC-003**
Scenario Name	**Register Vehicle**
Participating Actors	**Truck Owner, Admin**

Entry Conditions	Actor has an account and has the necessary **vehicle documents**
Flow Of Events	1. Actor **Logs In** to the system and clicks the **Register Vehicle** button 2. Actor is **redirected** to the **vehicle registration page** 3. Actor will be **choose** the **vehicle class**, **upload personal identification, drivers license, vehicle information document, vehicle registration certificate, vehicle insurance**, etc and **submit** 4. System will check for **unintelligible** or **corrupted** files 5. If successful, system will send the data for **admin review** 6. Admin will review the documents and **allow vehicle registrations** if all requirements are met 7. If successful, Actor has **successfully added** a **vehicle** to his account and can **operate** with it
End Conditions	1. Vehicle is **registered** to the account 2. Vehicle registry is **denied** and the **reason** is provided
Quality Requirements	1. Actor provides **legally** and **digitally valid** information and documents

2.6.1-4 Send KULI Request

Table 2.4: Send KULI Request

Scenario ID	**SC-004**
Scenario Name	**Send KULI Request**
Participating Actors	**Client / Call Center Assistant**
Entry Conditions	The actor is logged into the system
Flow Of Events	1. Actor clicks the **KULI Request** button and is redirected to the sending page 2. Actor inputs the **vehicle class of choice, destination, estimated luggage weight and volume, incentive tip amount, and other required inputs** and submit 3. System **validates input** and **calculates KULI price** 4. System searches and displays the **matched** and **recommended vehicle list** sorted by **proximity** and **driver rating** 5. Actor selects **preferred drivers** and sends request
End Conditions	1. Actor successfully **sends** KULI request 2. Actor does not **follow through** with the request 3. System does not find available vehicles on **standby**

Quality Requirements	The actor provides **honest** and **reasonable** information

2.6.1-5 Accept KULI Request

Table 2.5: Accept KULI Request

Scenario ID	**SC-005**
Scenario Name	**Accept KULI Request**
Participating Actors	**Truck Owner**
Entry Conditions	1. Actor is **Logged In** 2. Actor has an **active registered vehicle** 3. Actor is within a reasonable proximity to the request
Flow Of Events	1. System sends request to the fitting **Truck Owners** on **standby** 2. Actor **reviews** KULI request and **contacts** Client via contact info 3. If in agreement, Truck Owner **accepts** the KULI request
End Conditions	1. Actor contacts Client, is **in agreement** and **accepts** request 2. Actor contacts Client, is **not in agreement** and

	rejects request
Quality Requirements	Actor provides a **reasonable** argument

2.6.2. Use Case Model

Table 2.6: Use Case Model

Use Case ID	**Primary Actor**	**Use Case Name**	**Functionality**
UC-001	Admin, Client, Truck Owner, Call Assistant	Login/Register	Registration and Authentication of respective users
UC-002	Admin	Manage Users/Reports	View reports and Edit/Suspend/Ban/Remove/verify an account
UC-003	Truck Owner	Register/Manage Vehicle	Register Vehicle. Set currently active vehicle and update vehicle information.
UC-004	Client, Call Assistant	Create KULI Request	Create a KULI request based on preferences
UC-005	Truck	Handle KULI	Review, accept/ignore, cancel a KULI request

	Owner	Request	
UC-006	Truck Owner	Update KULI Status	Update an accepted KULI status
UC-007	Client	Rate Terminated KULI	Rate Truck Owner on a cancelled or completed KULI
UC-008	Client, Call Assistant	Report Truck Owner	Report truck owner misconduct
UC-009	Admin	Handle Vehicles	Add/Edit/Remove vehicle class and approve vehicle registry
UC-010	Call Assistant	Handle Assisted Booking	Allows customer service to create KULI requests on behalf of clients
UC-011	Call Assistant	Handle Hotline Ticker	Update ticket status from initiation to finalization
UC-012	TruckOwner	Confirm Payment	Confirm calculated payment completion

2.6.2.1. Use Case Diagram

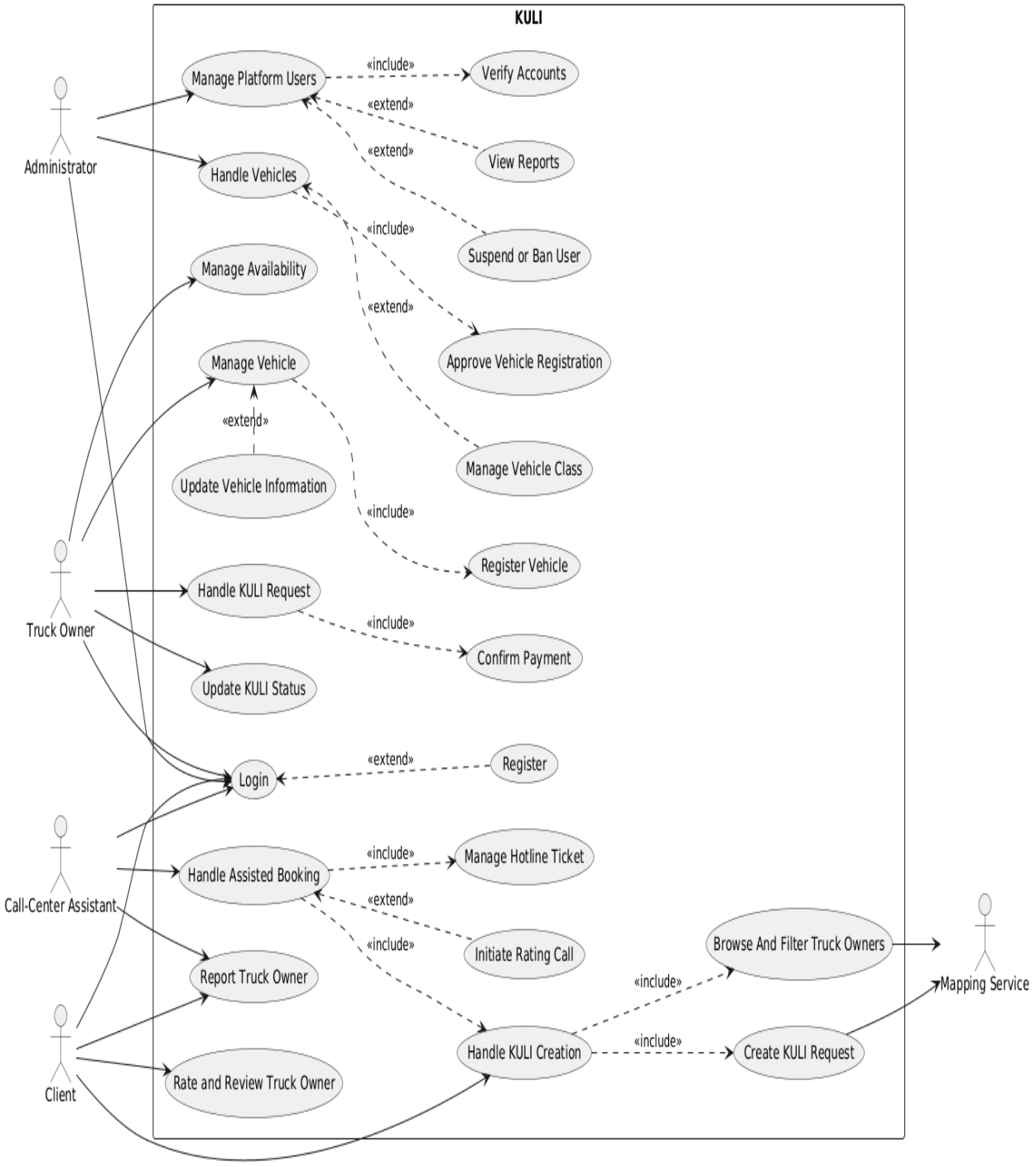


Figure 2.1: Use Case Diagram

2.3.6.2.2 Description Of Use Case Model

2.6.2.2-1: UC-001

Table 2.7: UC-001

Use Case Name	Login / Register
Corresponding FR	FR-1.1, FR-1.2, FR-1.3, FR-1.4
Description	Allows Clients, Truck Owners, Admins, and Call Assistants to create accounts or log in.
Actors	Client, Truck Owner, Admin, Call Assistant
Pre Conditions	● User has access to the system ● For login: account exists ● For registration: valid credentials available
Flow of Events	1. Select Login/Register 2. System displays form 3. Actor submits credentials 4. System validates 5. For login → authenticate 6. For registration → store new account 7. Redirect to dashboard
Post Conditions	● User authenticated or registered

	• System maintains session
Alternative Flows	• Invalid credentials → correction • Duplicate data → reject • Reset password option

2.6.2.1-2: UC-002

Table 2.8: UC-002

Use Case Name	Manage Users / Reports
Corresponding FR	FR-2.2, FR-8.4, FR-9.1
Description	Admin reviews reports and manages user accounts.
Actors	Admin
Pre Conditions	• Admin is logged in • Reports or accounts exist
Flow of Events	1. Admin selects management module 2. System shows users/reports 3. Admin selects entry 4. Admin applies action (verify/suspend/ban/edit/remove) 5. System saves changes

Post Conditions	● User state or report status updated
Alternative Flows	● Invalid action blocked ● Missing details → request more data

2.6.2.1-3: UC-003

Table 2.9: UC-003

Use Case Name	Register / Manage Vehicle
Corresponding FR	FR-2.1, FR 10.1
Description	Truck Owners register vehicles, set active vehicle, and update details.
Actors	Truck Owner
Pre Conditions	● Truck Owner logged in ● Valid documents available
Flow of Events	1. Open Vehicle Management 2. System shows vehicles 3. Submit new vehicle 4. System validates 5. Store vehicle

	6. Owner may set active vehicle or update info
Post Conditions	● Vehicle registered/updated ● Active vehicle set
Alternative Flows	● Invalid/missing documents ● Duplicate license plate

2.6.2.1-4: UC-004

Table 2.10: UC-004

Use Case Name	Create KULI Request
Corresponding FR	FR-3.1, FR-3.2, FR-4.1, FR-4.3, FR-5.1, FR-7.3
Description	Client or Call Assistant creates a new KULI request.
Actors	Client, Call Assistant
Pre Conditions	● Actor logged in ● Pickup/destination data available
Flow of Events	1. Select Create Request 2. Fill trip details 3. Submit request 4. System calculates estimate

	5. Stores request 6. Notifies Truck Owners
Post Conditions	● Request created ● Truck Owners alerted
Alternative Flows	● Missing fields ● No available vehicles

2.6.2.1-5: UC-005

Table 2.11: UC-005

Use Case Name	Handle KULI Request
Corresponding FR	FR-10.2, FR-7.3, FR-2.3
Description	Truck Owners view, accept, ignore, or cancel KULI requests.
Actors	Truck Owner
Pre Conditions	● Truck Owner logged in ● Pending requests exist
Flow of Events	1. Open inbox 2. System shows requests 3. Owner selects one

	4. Accept/ignore/cancel 5. System updates status 6. Notify client if accepted
Post Conditions	● Request status updated
Alternative Flows	● Another owner accepts first ● Invalid action

2.6.2.1-6: UC-006

Table 2.12: UC-006

Use Case Name	Update KULI Status
Description	Truck Owner updates a KULI state during the trip.
Corresponding FR	FR-6.1, FR-6.2, FR-10.3 FR 7.3
Actors	Truck Owner
Pre Conditions	● KULI accepted ● Owner logged in
Flow of Events	1. Open active KULI 2. View current status 3. Update status (in-progress/arrived/completed)

	4. System saves 5. Notify Client
Post Conditions	● Status updated
Alternative Flows	● Invalid status change

2.6.2.1-7: UC-007

Table 2.13: UC-007

Use Case Name	Rate Terminated KULI
Corresponding FR	FR-8.1, FR-8.2
Description	Clients rate a completed or cancelled KULI.
Actors	Client
Pre Conditions	● KULI completed/cancelled ● Client logged in
Flow of Events	1. Select KULI 2. System displays rating form 3. Client submits rating 4. System saves 5. Recalculate owner rating

Post Conditions	● Rating stored
Alternative Flows	● Missing rating value

2.6.2.1-8: UC-008

Table 2.14: UC-008

Use Case Name	Report Truck Owner
Corresponding FR	FR-8.4
Description	Client or Assistant submits a misconduct report.
Actors	Client, Call Assistant, Admin
Pre Conditions	● KULI exists ● Actor logged in
Flow of Events	1. Select report option 2. Provide incident details 3. Submit report 4. System saves 5. Admin reviews and acts
Post Conditions	● Report logged ● Admin decision applied

Alternative Flows	● Incomplete report ● Invalid report rejected

2.6.2.1-10: UC-09

Table 2.16: UC-09

Use Case Name	Handle Vehicles
Description	Admin handles vehicle categories and approvals.
Corresponding FR	FR-2.2, FR-9.1
Actors	Admin
Pre Conditions	● Admin logged in ● Vehicle entries exist
Flow of Events	1. Open vehicle management 2. View pending approvals 3. Select vehicle 4. Approve/edit/remove 5. Save changes
Post Conditions	● Vehicle registry updated

Alternative Flows	● Invalid documents ● Missing details

2.6.2.1-11: UC-010

Table 2.17: UC-010

Use Case Name	Handle Assisted Booking
Description	Call Assistant books a KULI on behalf of a client.
Corresponding FR	FR-7.2, FR-3.1, FR-5.1
Actors	Call Assistant
Pre Conditions	● Assistant logged in ● Client info available
Flow of Events	1. Receive call 2. Open assisted booking page 3. Enter client and trip data 4. System searches truck owners and calculates cost 5. Assistant confirms selection with client 6. Request stored and notifications sent
Post Conditions	● Assisted KULI created

Alternative Flows	● Incomplete client data ● Request canceled by client

2.6.2.1-12: UC-011

Table 2.18: UC-011

Use Case Name	Handle Hotline Ticket
Description	Assistant updates ticket progress during assisted bookings.
Corresponding FR	FR-7.2, FR-6.1
Actors	Call Assistant
Pre Conditions	● Ticket exists ● Assistant logged in
Flow of Events	1. Open ticket dashboard 2. Select ticket 3. Update status 4. System saves 5. Notify Client
Post Conditions	● Ticket status updated
Alternative Flows	● Closed ticket cannot be edited

2.6.2.1-13 UC-012

Table 2.19: UC-012

Use Case Name	Confirm Payment
Description	Truck Owner confirms payment after KULI completion.
Corresponding FR	FR-5.2, FR-5.3, FR-7.3
Actors	Truck Owner
Pre Conditions	● KULI completed ● Truck Owner logged in
Flow of Events	1. Select completed KULI 2. View cost 3. Confirm payment 4. System records it 5. Client notified
Post Conditions	● Payment status updated
Alternative Flows	● Disputed amount → notify Admin

2.6.3 Object Model

2.6.3.1 Data Dictionary

Table 2.20: Data Dictionary

Object / Entity	Type	Attributes / Methods	Description
User	Entity	userID, role, name, email, phone, password, address	Represents a user involved in the system
Role	Entity	roleID, name	Represents user role
Vehicle	Entity	vehicleID, ownerID, type, capacity, licensePlate, status, insurance, documents	Represents a truck registered in the system
KULI	Entity	requestID, clientID, ownerID, pickupLocation, pickupTime, destination, loadDetails, status, tip, Cost	Represents a transportation request
Ticket	Entity	ticketID, requestID, assignedAssistant, status	Represents a hotline ticket when no Truck Owner is immediately available
Report	Entity	reportID, reporterID, truckOwnerID, description, status	Represents complaints about Truck Owners or system issues

Location	Entity	locationID, latitude, longitude, address	Represents geospatial data for pickup and destination
Notification	Entity	notificationID, userID, message	Represents notifications sent to users

2.6.3.2 Class Modeling

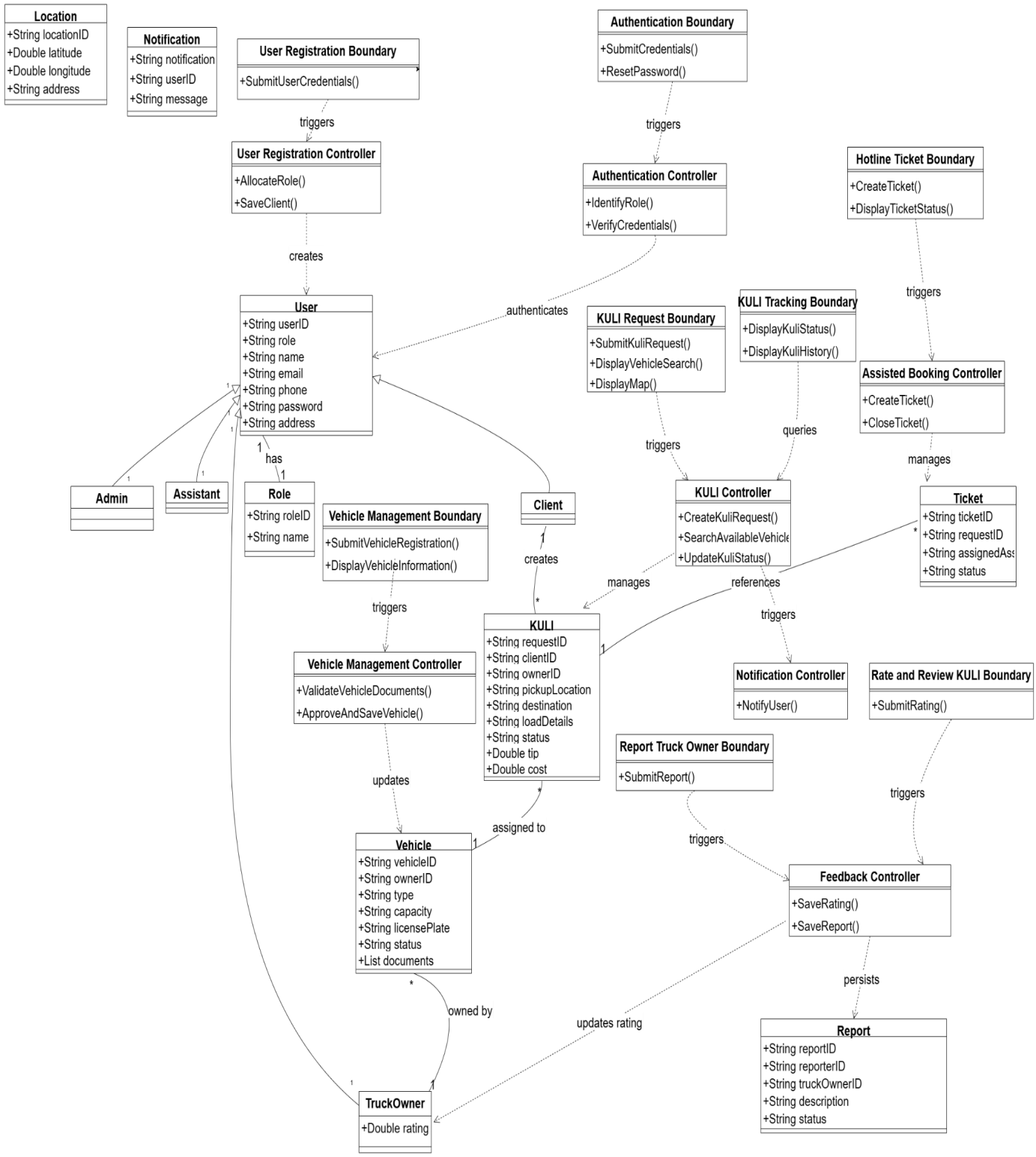


Figure 2.2: Class Model diagram

2.3.6.4 Dynamic Modeling

This section will illustrate the dynamic model of the system with sequence diagrams, activity charts, and state charts.

2.3.6.4.1 Sequence Diagrams

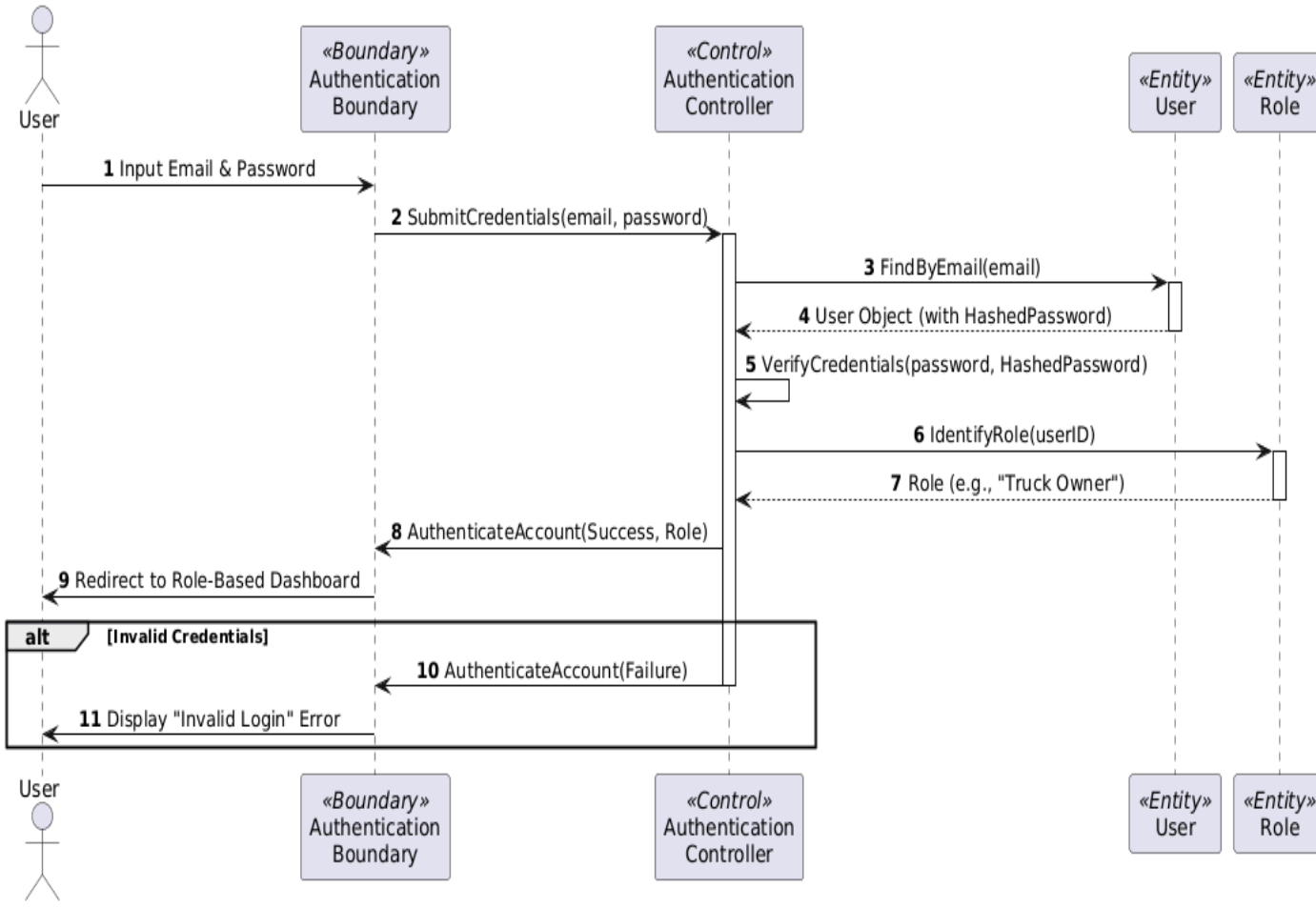


Figure 2.3: Authentication Diagram

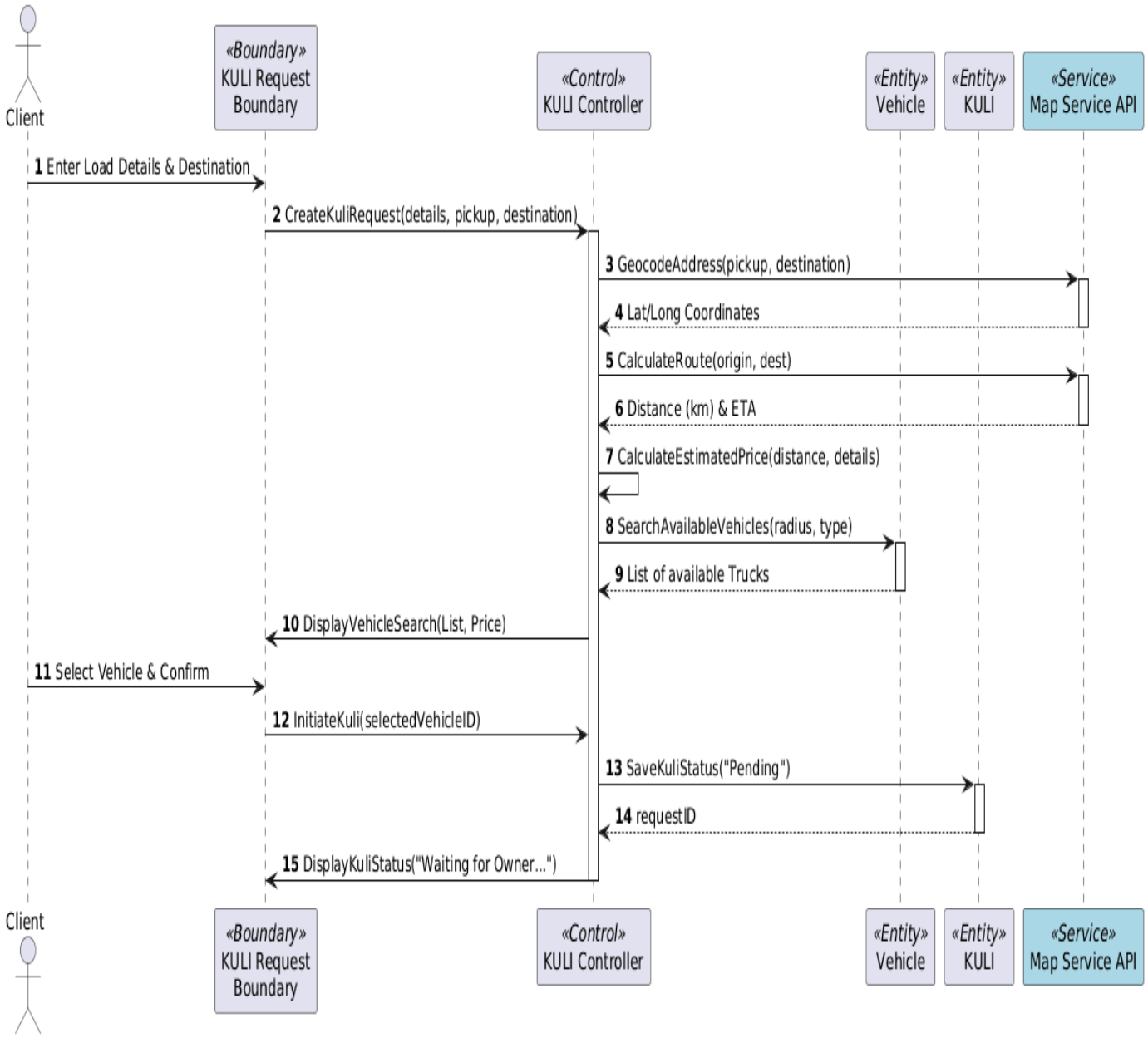


Figure 2.4: Kuli Request

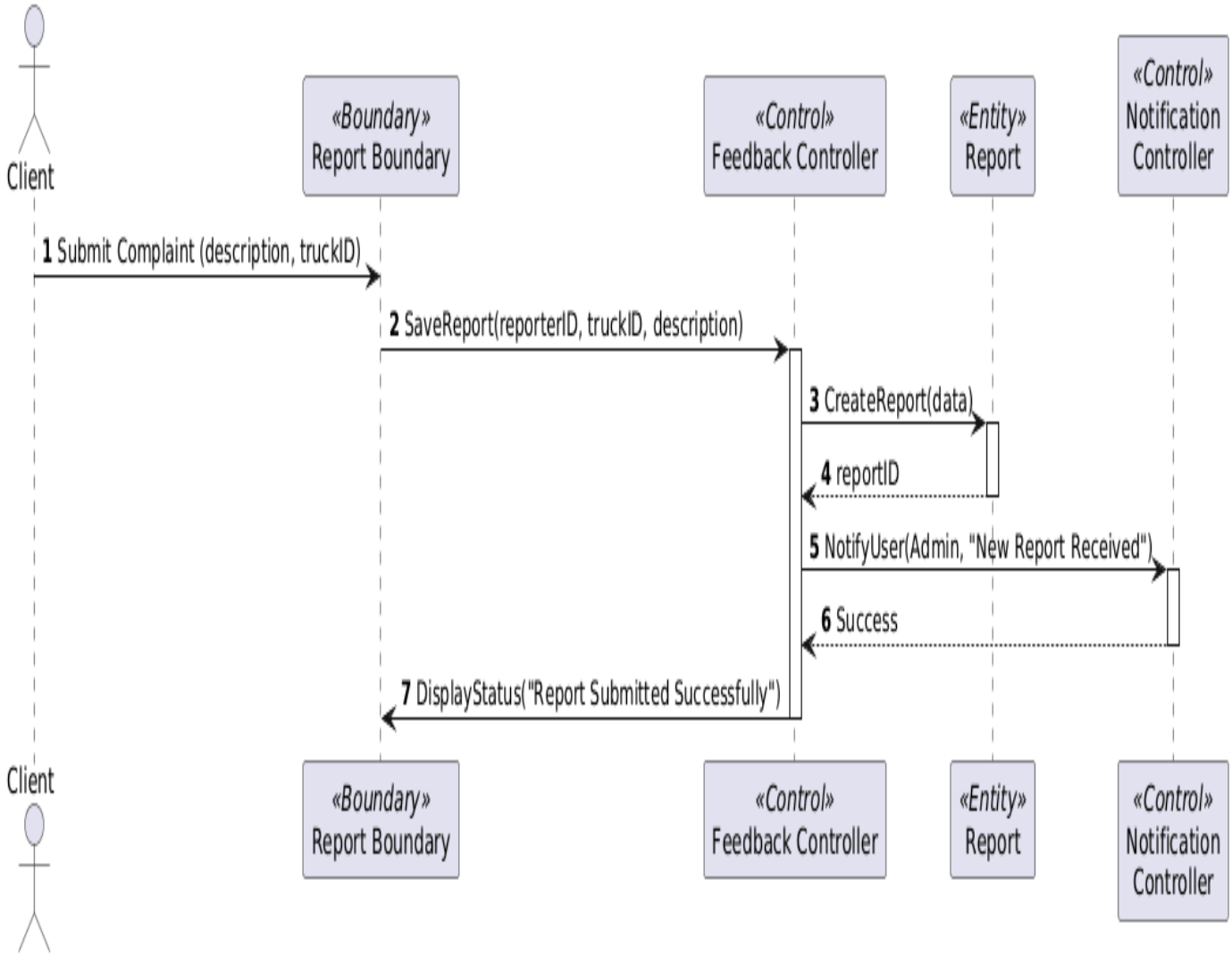


Figure 2.5: Report Truck Owner

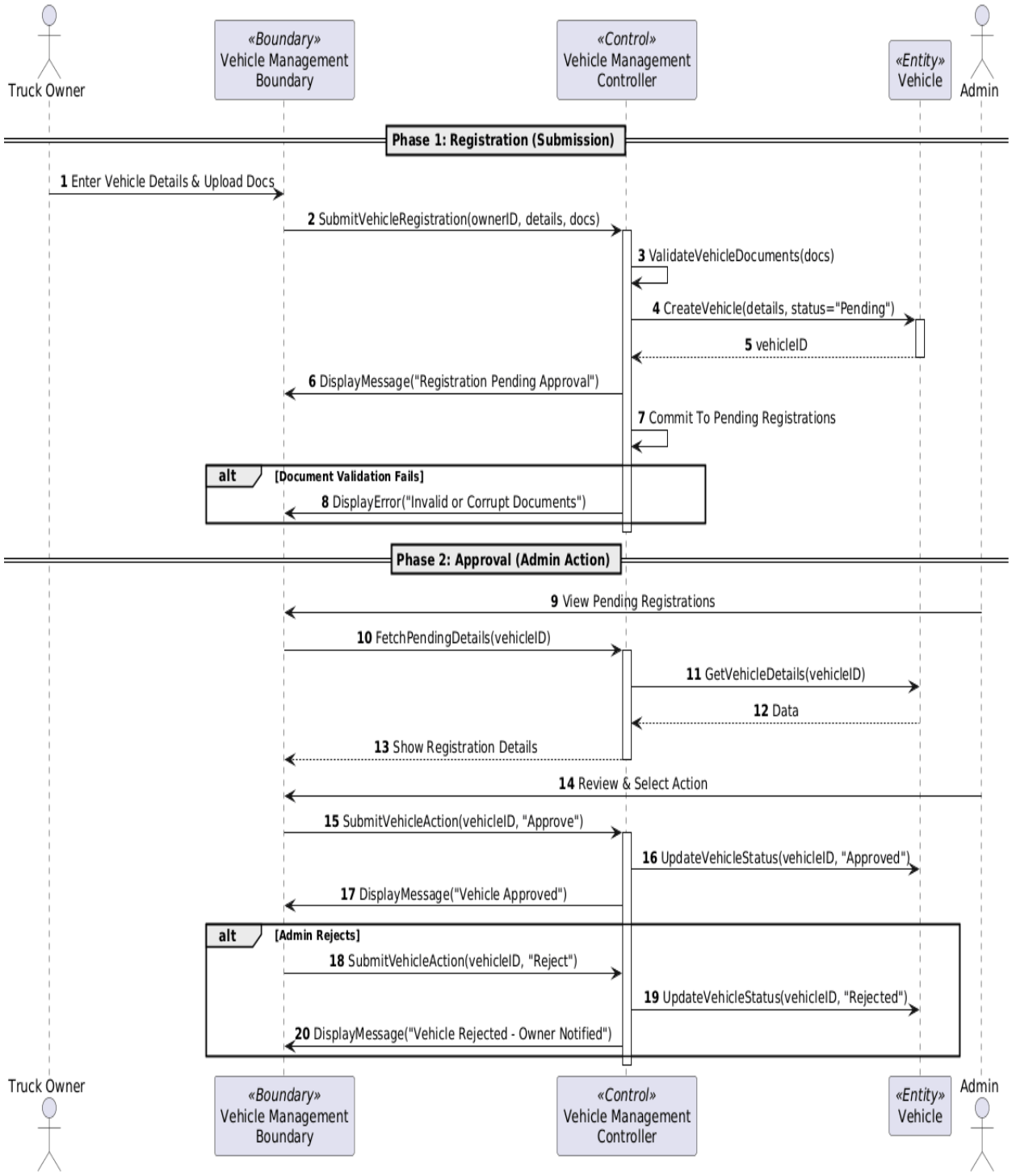


Figure 2.6: Vehicle Approval

2.6.4.2 Activity Diagrams

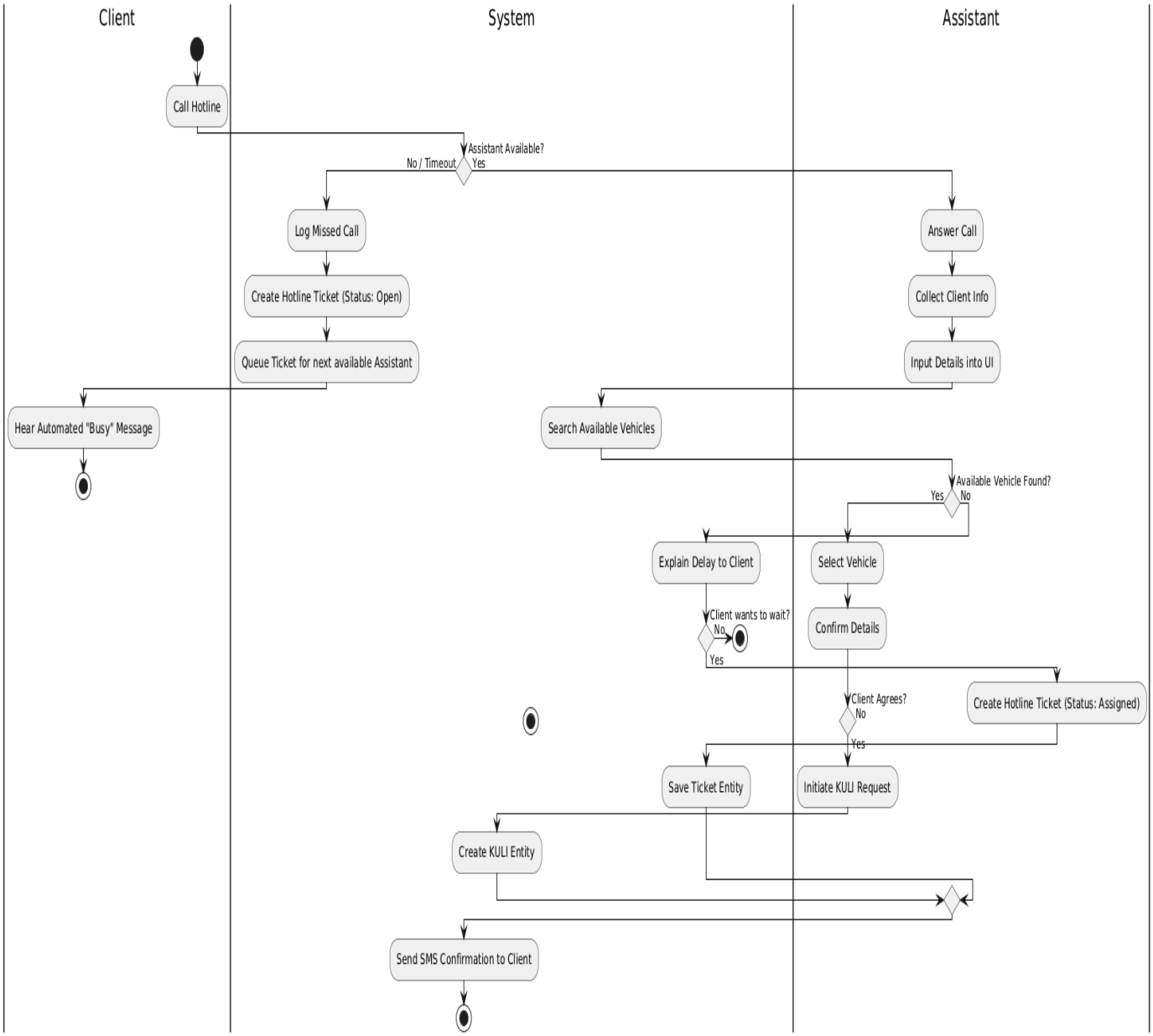


Figure 2.7: Assisted Booking

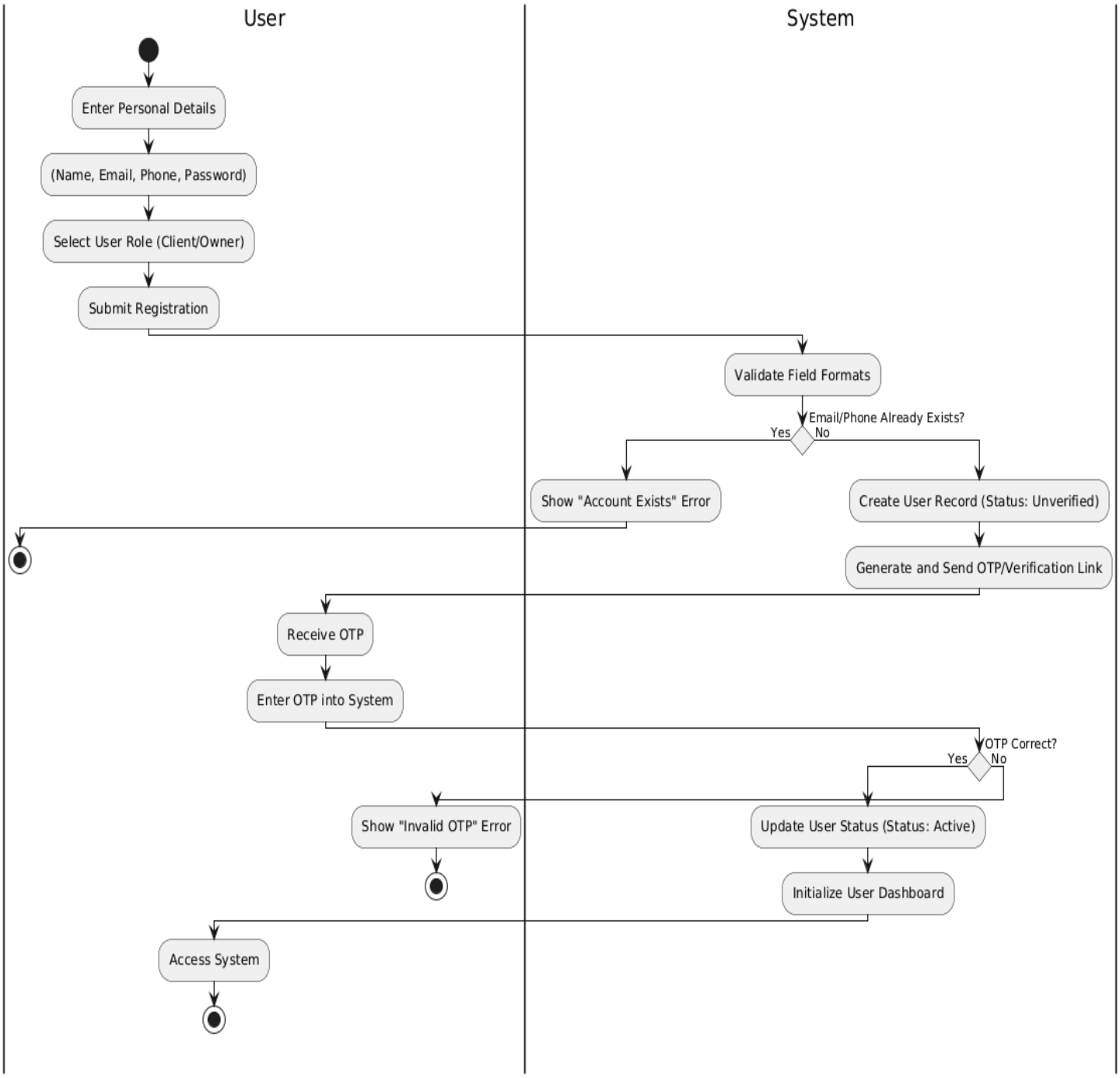


Figure 2.8: Registration

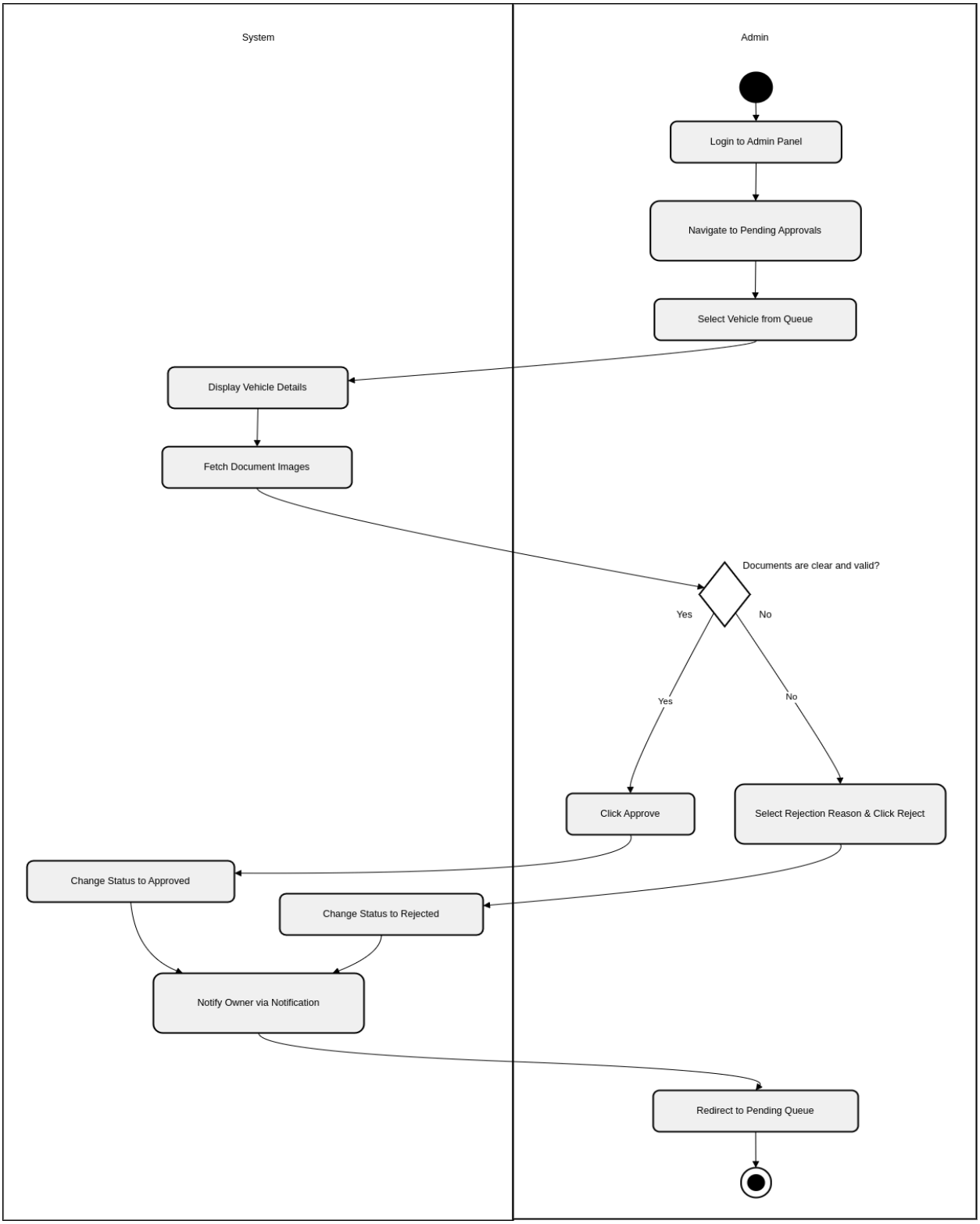


Figure 2.9: Vehicle Approval

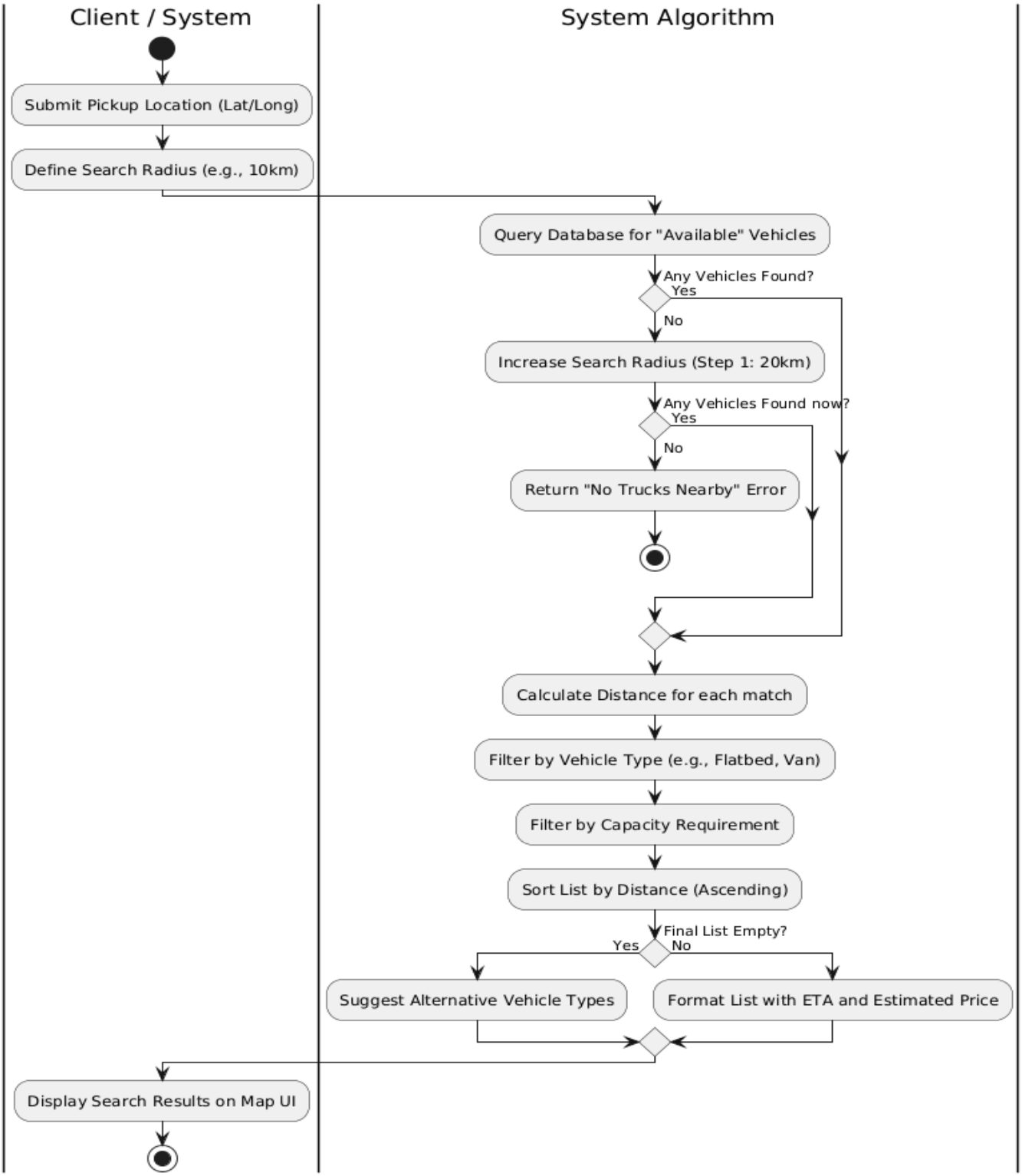


Figure 2.10: Search Nearby Vehicle

2.6.4.3 State Charts

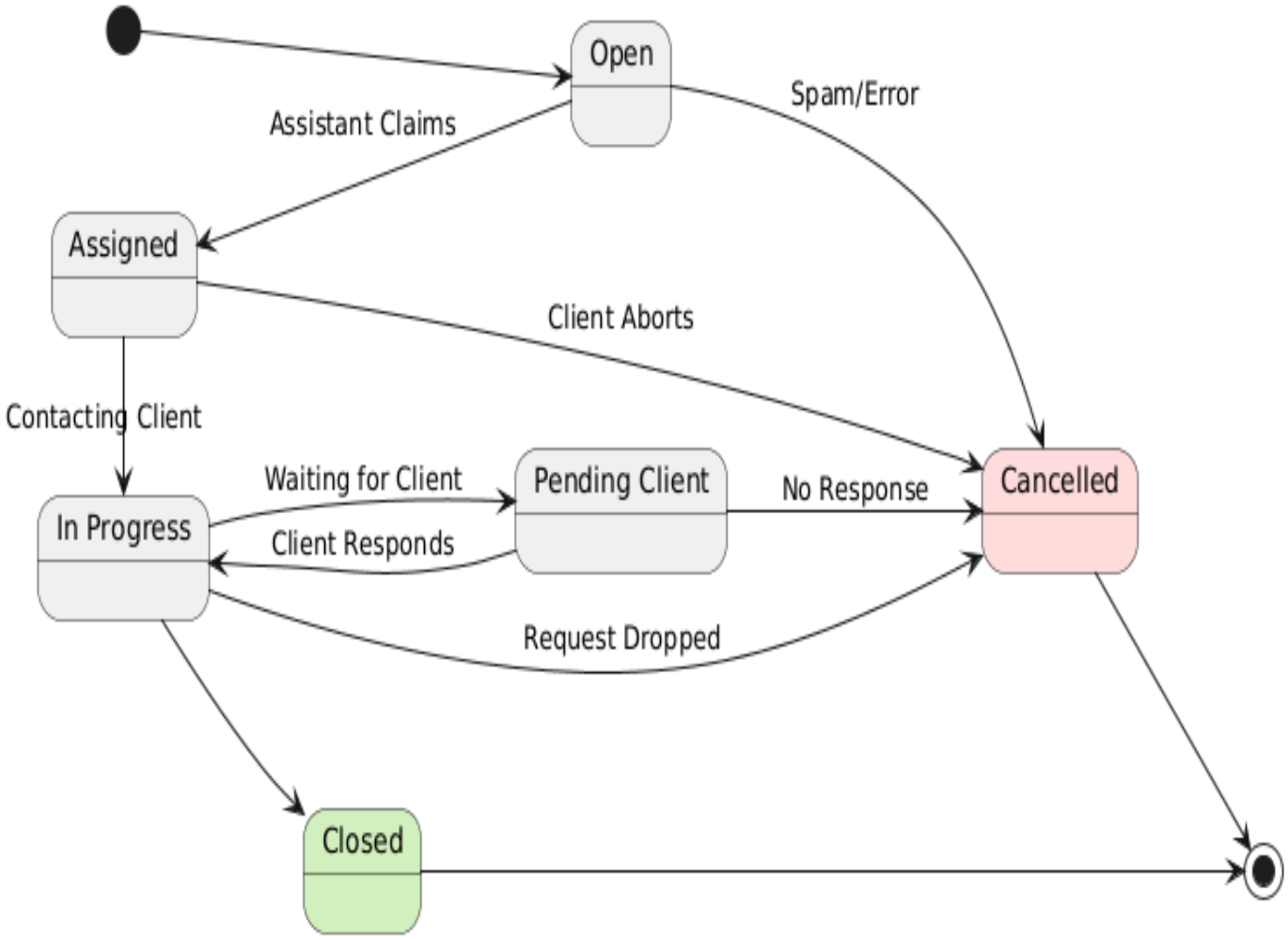


Figure 2.11: Ticket State Chart

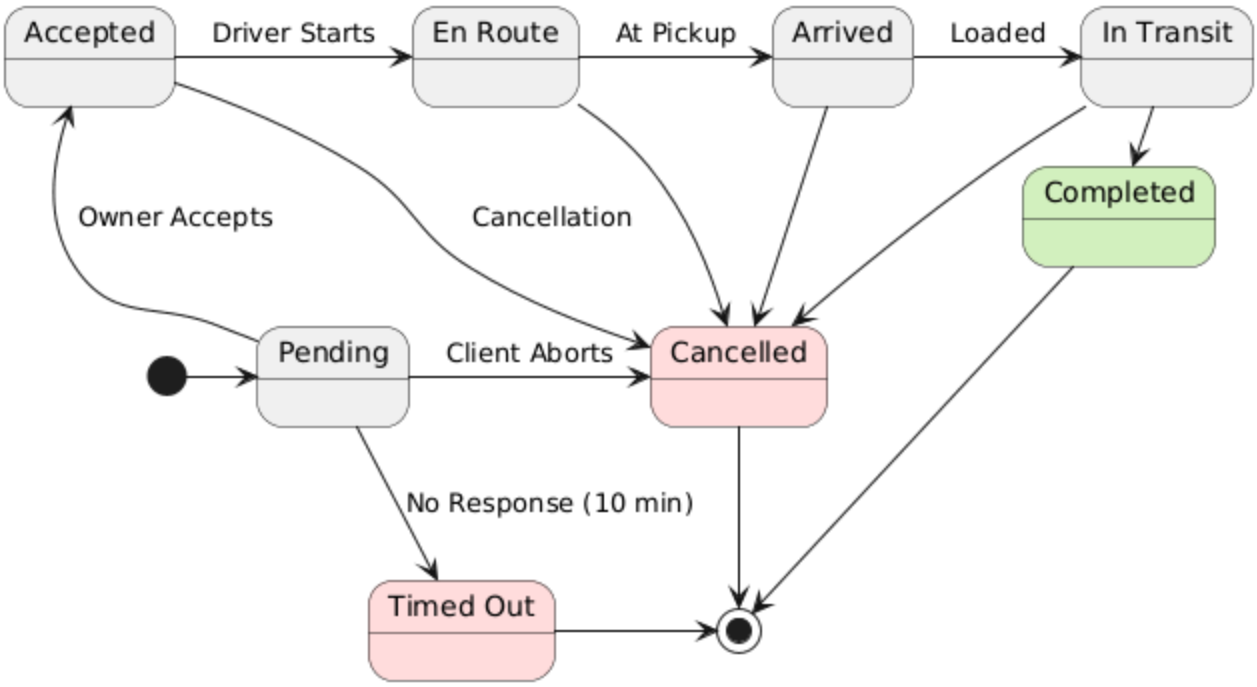


Figure 2.12: Kuli State Chart

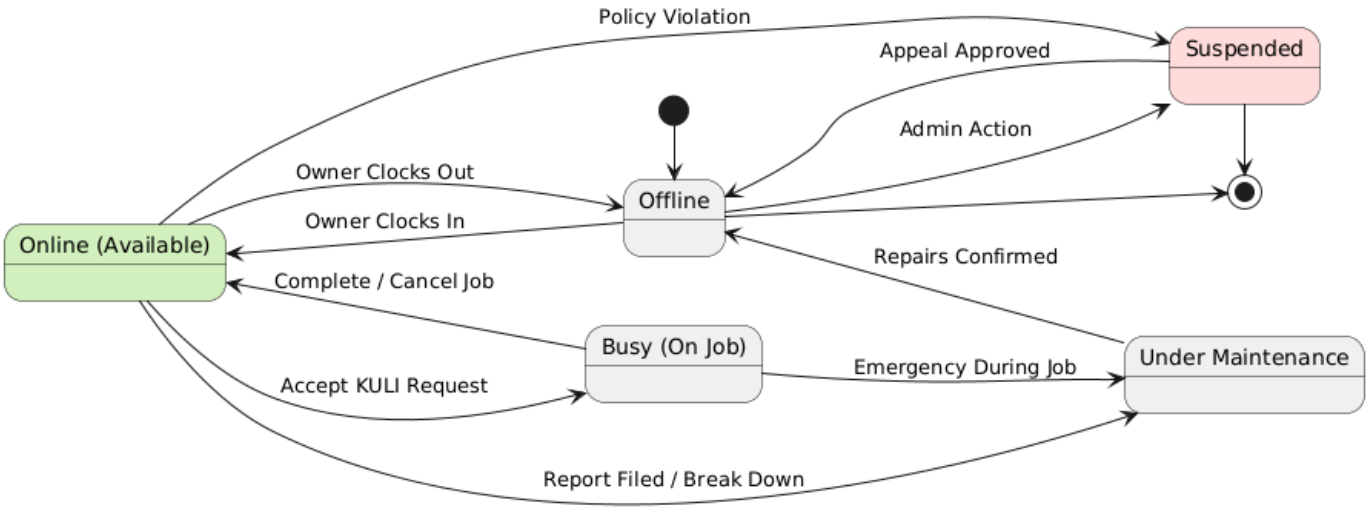


Figure 2.13: Vehicle State Chart

2.6.5 Client Interface

Overview of UI style and direction

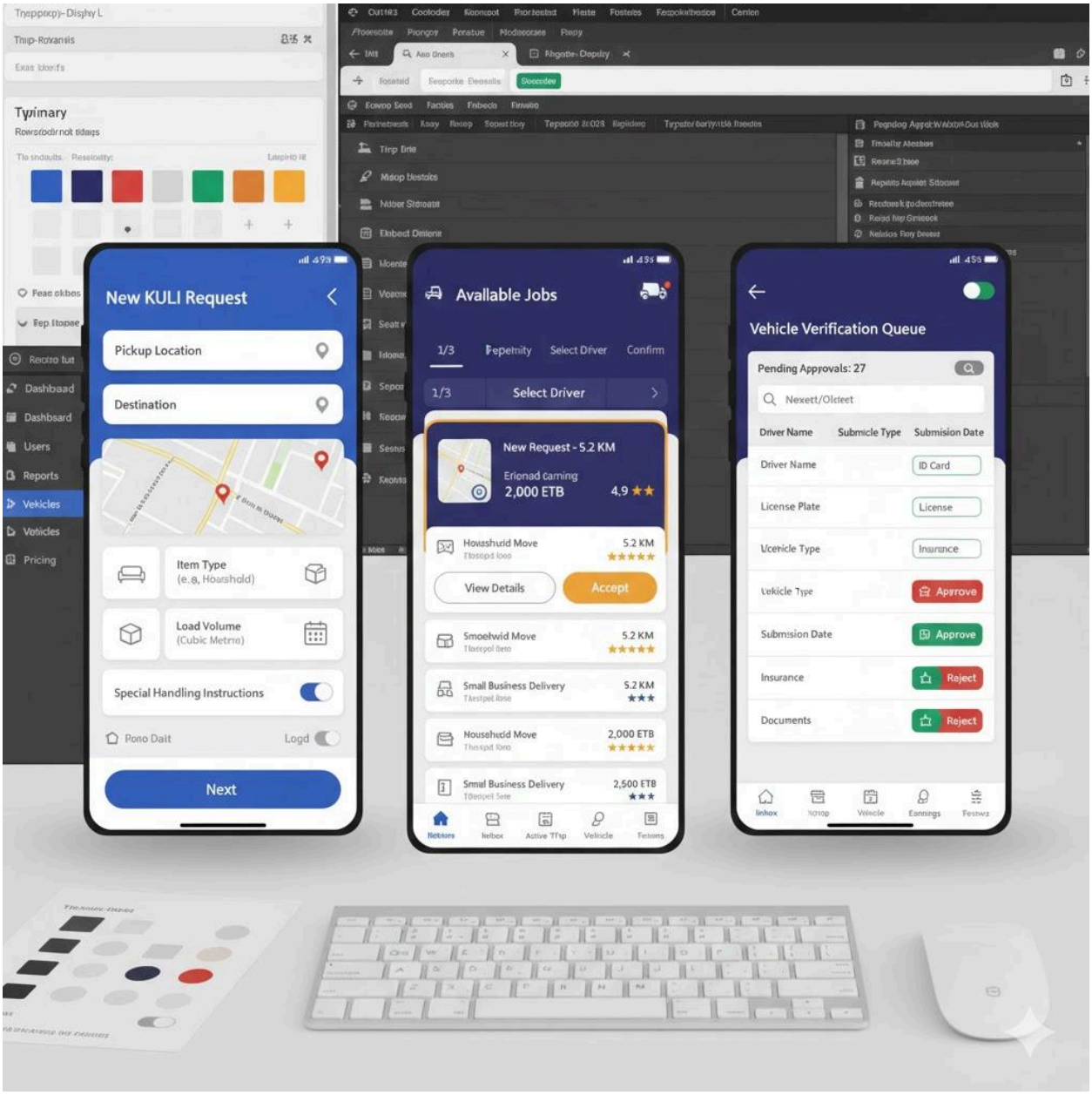


Figure 2.14: UI Template

3. SYSTEM DESIGN

3.1. Introduction

This chapter defines the technical transition from requirement analysis to system implementation for the **Kuli P2P Logistics Platform**. The design translates the functional needs—such as on-demand truck booking and assisted hotline services—into a structured, three-tier architecture.

Traceability & Constraints: The design is directly traced to the requirements identified in Chapter 2, specifically addressing the need for proximity-based matching and multi-role access. Key constraints impacting this architecture include the need for a low-cost initial deployment and the exclusion of real-time GPS/Payment gateways in favor of a manual status-update workflow.

Design Strategy Overview:

- **Subsystem Decomposition:** The backend is organized into 8 domain-specific modules (IAM, UA, TR, BP, LO, PM, FD, SU) to ensure high maintainability.
- **Hardware/Software Mapping:** Use of standard smartphones and workstations connecting to a centralized Node.js application server.
- **Persistent Data Management:** Utilization of MongoDB for flexible document storage and geospatial indexing.
- **Access Control & Security:** Implementation of Role-Based Access Control (RBAC) managed by the Identity & Access module.

3.2. Current Software Architecture

The system being replaced is a **manual, informal logistics network**. Currently, truck owners and clients rely on word-of-mouth, physical bulletin boards, and unmanaged phone calls.

- **Architecture Type:** Ad-hoc / Decentralized.
- **Limitations:** This "architecture" lacks a centralized data repository, leading to non-transparent pricing, lack of vehicle verification, and no formal record-keeping. The proposed system replaces this fragmented workflow with a unified digital engine.

3.3. Proposed Software Architecture

3.3.1. Overview

The proposed architecture for the Kuli platform is a **Layered Modular Monolith**. This bird's-eye view shows a three-tier structure comprising a Client Tier (Mobile and Web), an Application Tier (Node.js Backend), and a Data Tier (MongoDB and Archival Storage).

Assignment of Functionality: The core business logic is encapsulated within the **Application Tier** and distributed across the following subsystems:

- **Identity & Access (IAM):** Secures the system via authentication and RBAC.
- **User Accounts (UA):** Manages profiles for Clients, Owners, and Staff.
- **Truck Registry (TR):** Handles vehicle enrollment and compliance documents.
- **Booking & Pricing (BP):** Manages the marketplace, pricing, and truck discovery.
- **Live Operations (LO):** Orchestrates active trip states and manual status logging.
- **Financial Settlement (PM):** Generates invoices and tracks cash-based transactions.
- **Feedback & Disputes (FD):** Manages ratings and administrative mediation.
- **System Utilities (SU):** Provides notification dispatch and audit logging.

This modularity ensures that while the system is deployed as a single unit for simplicity, it remains **scalable by design**, allowing individual modules to be extracted or expanded as the platform grows.

Architectural Style and Rationale

The chosen architectural style is the **Modular Monolith**. This style was selected over a traditional "Big Ball of Mud" monolith or a complex "Microservices" architecture for the following reasons:

- **Rationale for Modularity:** Unlike a standard monolith where all code is tightly coupled, a modular monolith enforces boundaries. This allows the team to work on the "Financial Settlement" module without risking side effects in the "Live Operations" module, satisfying the need for **system reliability**.

- **Justification for the Monolithic Deployment:** At this stage of the project, microservices would introduce unnecessary network latency and DevOps complexity (orchestration, service discovery, etc.). A single deployment unit simplifies testing and reduces infrastructure costs while the product-market fit is being established.
- **Future-Proofing (Scalability):** Because the subsystems are logically separated and communicate through defined internal APIs, the architecture provides a clear path to scalability. If a specific module (like Booking & Pricing) becomes a bottleneck, it can be extracted into a standalone service with minimal refactoring.
- **Consistency:** A unified data tier (MongoDB) ensures strong data consistency for logistics transactions, which is harder to achieve in distributed architectural styles.

System Scope & Constraints

To maintain a lightweight core, the architecture focuses on essential coordination. Real-time GPS telemetry and third-party payment gateways are excluded from the current scope. The modular structure ensures these can be added as external service integrations in future versions without altering the base architecture.

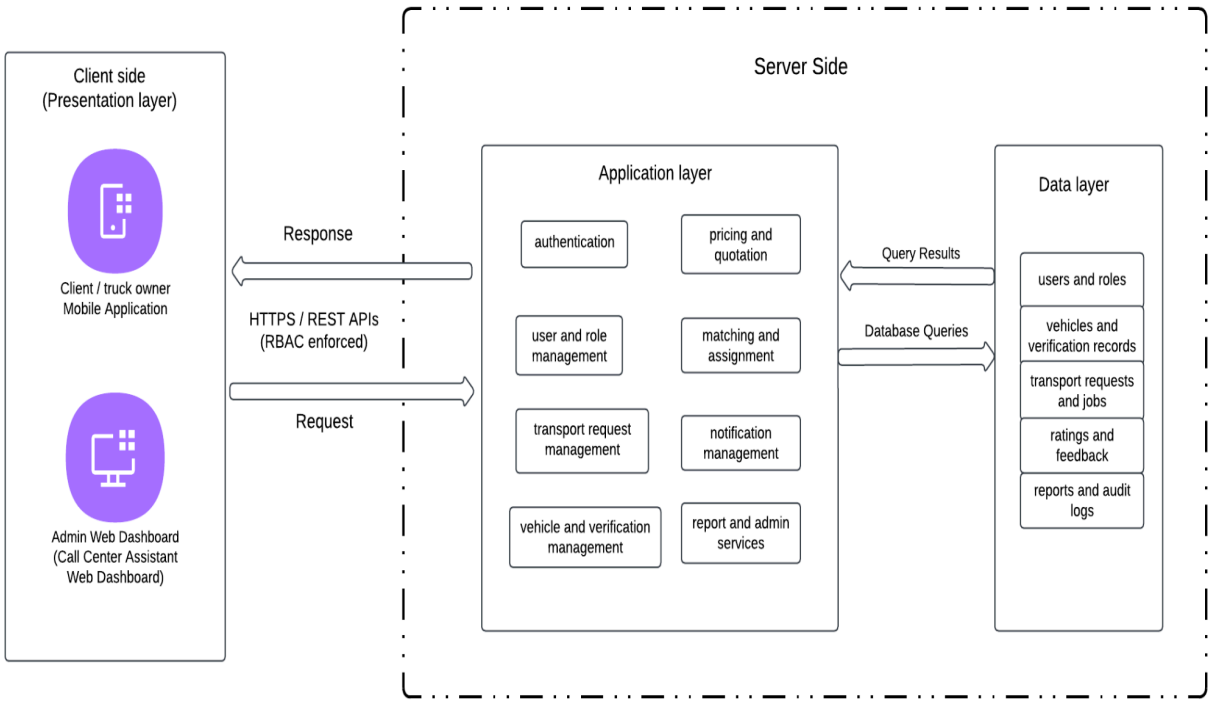


Figure 3.1: Architectural design diagram

3.3.2. Subsystem Decomposition

The KULI system is composed of the following major subsystems, each having dedicated responsibilities:

1. Access & Gateway (AG): Serves as the system's entry point, managing request routing, API gateway logic, protocol transformation, rate limiting, and cross-platform interface normalization.

2. Identity Provider (ID): Handles core security protocols, including credential management, multi-factor authentication (MFA) via OTP, secure session token issuance, and role-based access control (RBAC) enforcement.

3. User Accounts (UA): Manages the lifecycle of user digital personas, including multi-role profile metadata, saved preferences, billing/payout account details, and administrative account oversight.

4. Truck Registry (TR): Orchestrates the supply-side ecosystem, managing vehicle enrollment, document archival for compliance, administrative verification workflows, and real-time fleet availability status.

5. Booking & Pricing (BP): Functions as the marketplace engine, handling logistics request validation, algorithmic pricing based on distance and vehicle class, and proximity-based truck discovery and matching.

6. Live Operations (LO): Manages active trip states through a synchronized state machine, facilitating real-time GPS telemetry, in-transit milestone logging, and context-bound messaging between participants.

7. Financial Settlement (PM): Governs the fiscal integrity of the system, managing transaction recording, automated invoice generation, payout scheduling for owners, and tamper-proof financial auditing.

8. Feedback & Disputes (FD): Oversees platform quality and integrity, managing multi-criteria rating aggregation, text review moderation, dispute filing, and administrative mediation workflows.

9. System Utilities (SU): Provides cross-cutting infrastructure services, including a centralized notification dispatch hub (SMS/Email/Push) and a system-wide audit logging service for compliance and health monitoring.

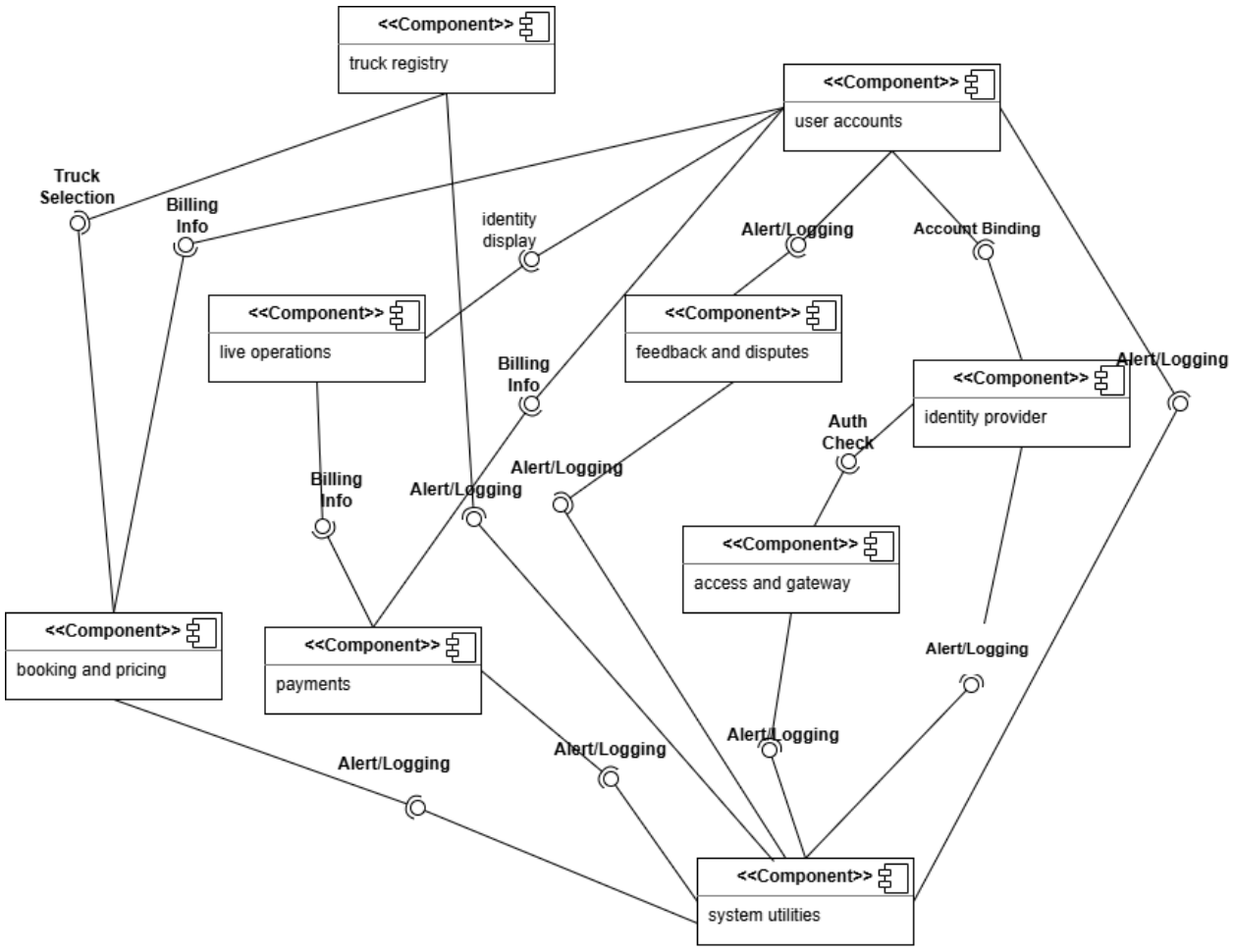


Figure 3.2: Component Diagram

3.3.3. Hardware-Software Mapping

The **Kuli** platform follows a client-server model built on a **Node.js modular monolith**. Users interact with the system via a **React Native mobile app** (Android/iOS), while administrators utilize a **React-based web dashboard** on standard workstations. The backend leverages **REST/JSON** for transactional logic and in the future(later versions), **WebSockets** for real-time **GPS telemetry** within the Live Operations module. Data is persisted in **MongoDB** using **geospatial indexing** for efficient truck discovery. To maintain a lightweight core, the architecture integrates **off-the-shelf APIs** for mapping and routing and **Supabase** for authentication.

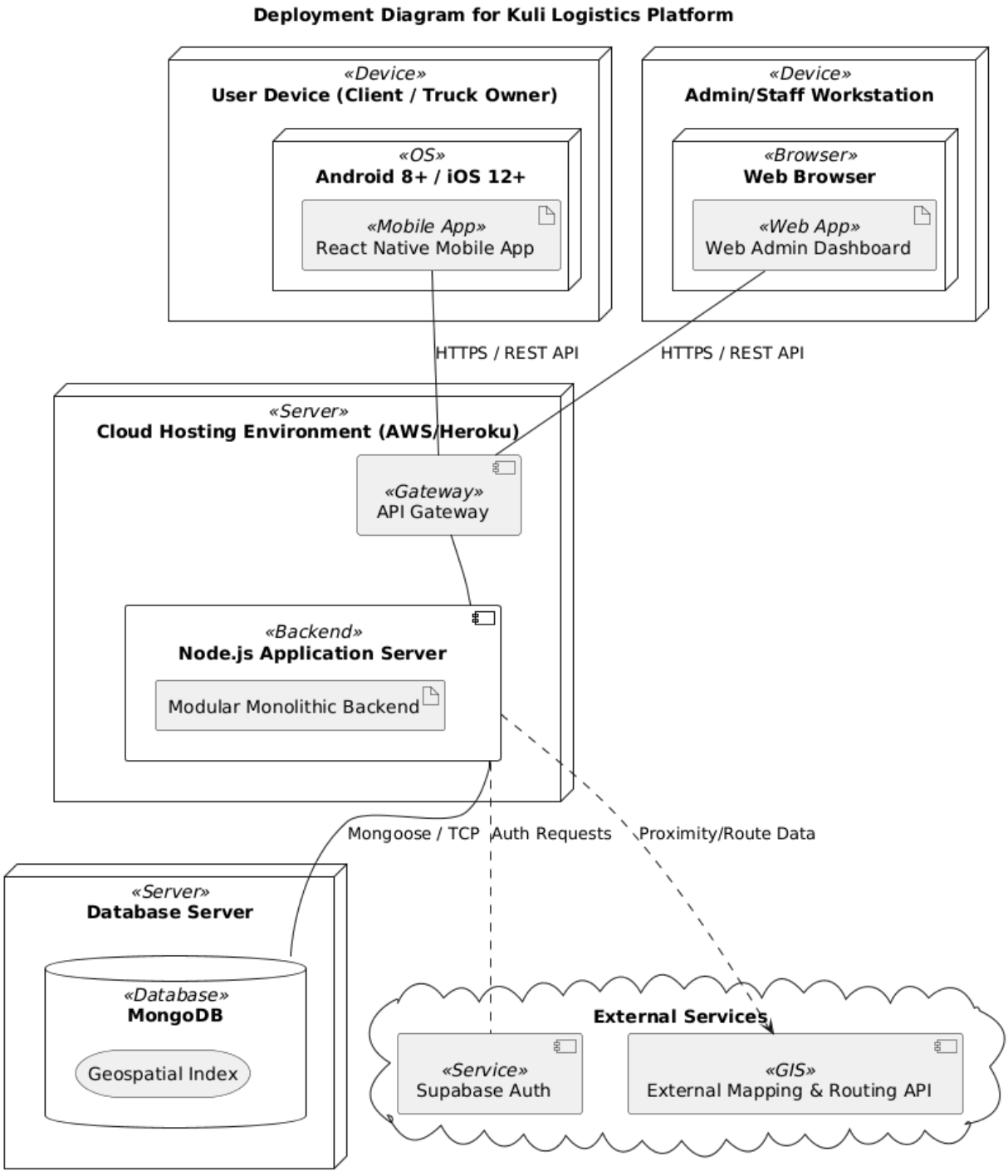


Figure 3.3: Deployment Diagram

3.3.4. Persistent Data Management

3.3.4.1. Description of Data Schemes

Based on the system's core entities, the data model is organized into several key document collections:

- **Users**: Stores digital personas for all roles (Client, Truck Owner, Assistant, Admin), including authentication credentials, contact details, and role-based metadata.
- **Vehicles**: Manages the truck registry, linking vehicles to their owners and storing compliance data such as license plates, capacity, and insurance documents.
- **Kuli Requests (Trips)**: The central transactional entity that tracks the lifecycle of a logistics request from creation to completion, linking clients to truck owners.
- **Locations**: Stores geospatial coordinates (latitude and longitude) for pickup and destination points to facilitate distance calculations and proximity matching.
- **Hotline Tickets**: Manages the state of assisted bookings created by call-center assistants on behalf of users.
- **Reports & Notifications**: Tracks user feedback, disputes, and the history of system-generated alerts.

3.3.4.2. Selection of Database

The platform utilizes **MongoDB** as its primary persistent storage engine. This choice is driven by the following factors:

- **Geospatial Indexing**: MongoDB's native support for 2dsphere indexes is critical for the "proximity-based listing mechanism," allowing the system to efficiently find available truck owners near a client's pickup location.
- **Schema Flexibility**: As a document-oriented database, MongoDB allows for evolving data structures, which is essential for handling diverse vehicle types and future integration of digital payment gateways.
- **Scalability**: Its distributed nature supports the project's long-term goal of scaling beyond the initial Addis Ababa scope.

3.3.4.3. Description of Database Encapsulation

The database is encapsulated using an **Object-Document Mapper (ODM)**, specifically **Mongoose**, within the Node.js backend. This provides:

- **Data Validation**: Enforcing schema rules at the application level to ensure that required fields like licensePlate or pickupLocation are present before persistence.
- **Abstraction**: Subsystems interact with high-level JavaScript objects rather than writing raw database queries, which maintains the integrity of the modular monolith architecture.
- **Security**: Direct database access is restricted; all data operations must pass through the Backend Engine, which enforces **Role-Based Access Control (RBAC)** as managed by the Identity & Access (IAM) subsystem.

3.3.4.3. Object Diagram

Since the Kuli platform utilizes a document-oriented database (MongoDB), the following object diagrams serve as representational snapshots of the system's persistent data state.

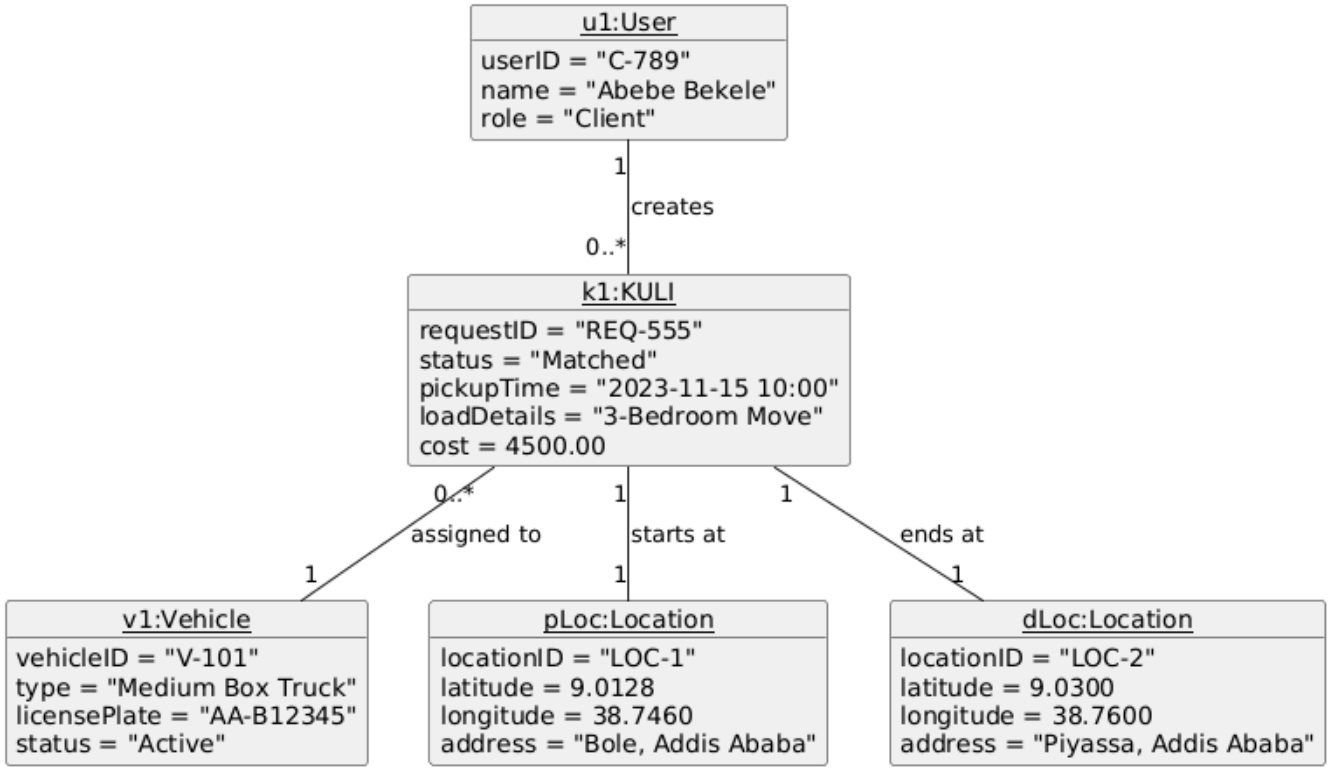


Figure 3.4: Snapshot of a Matched Service Diagram

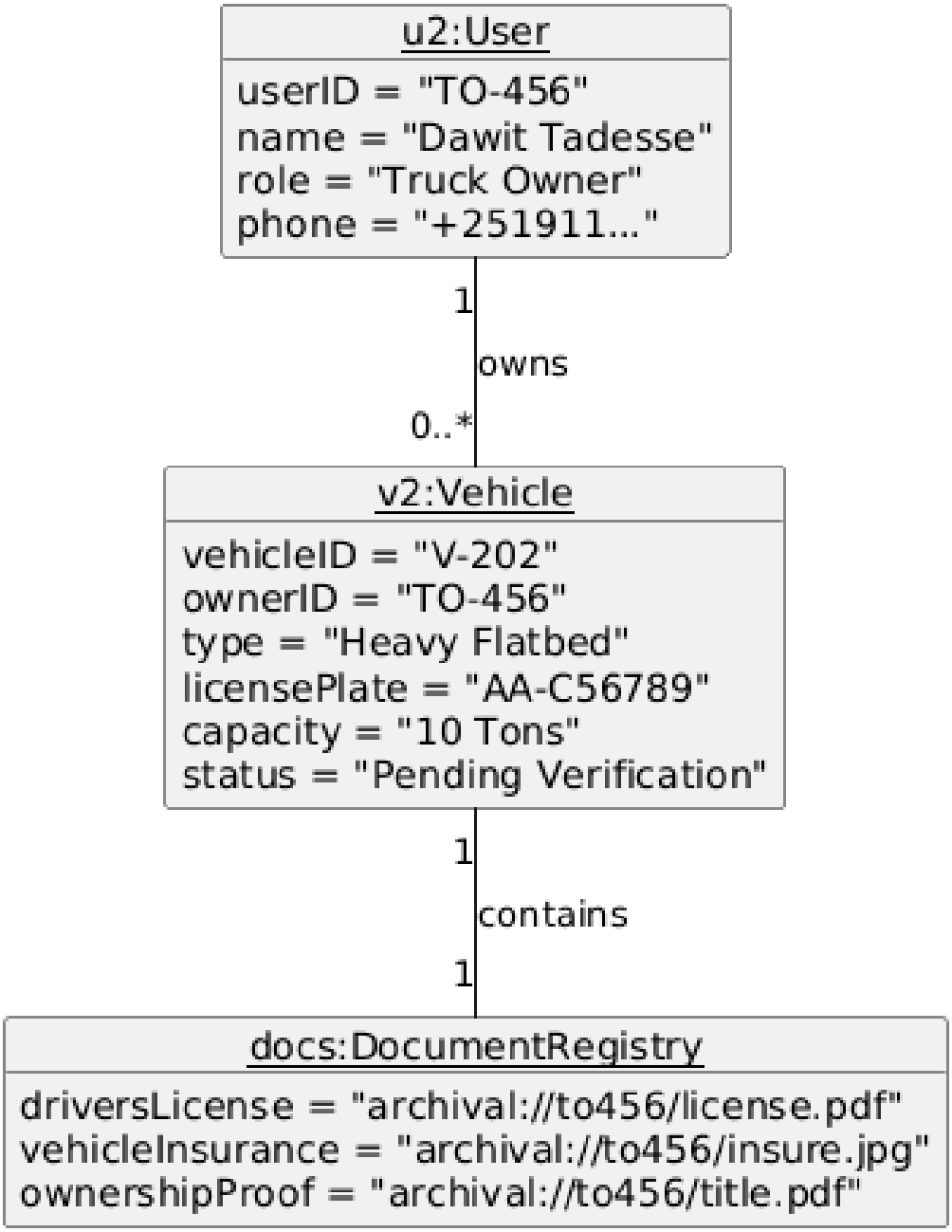


Figure 3.5: Snapshot of Vehicle Onboarding Diagram

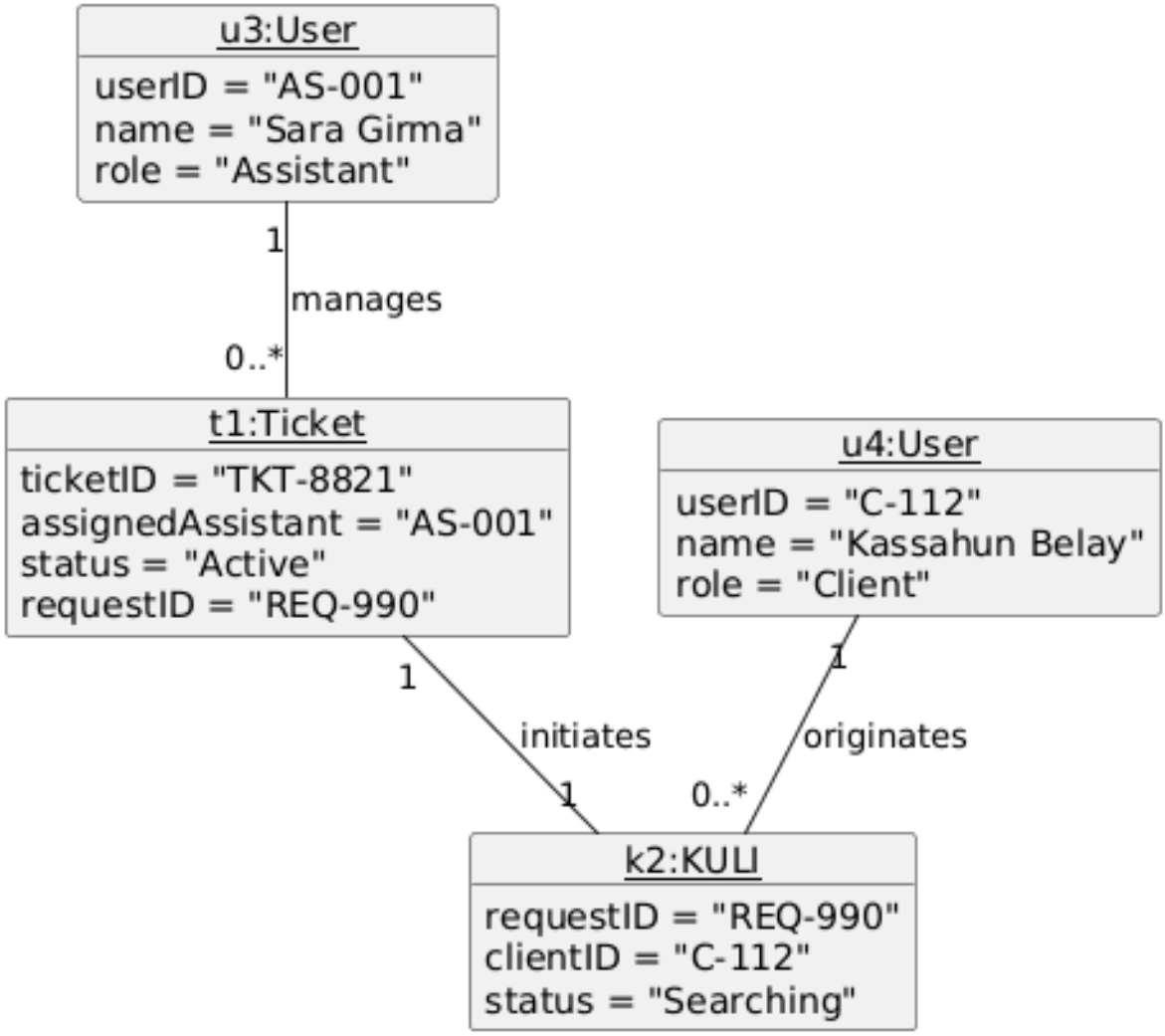


Figure 3.6: Snapshot of Assisted Booking Diagram

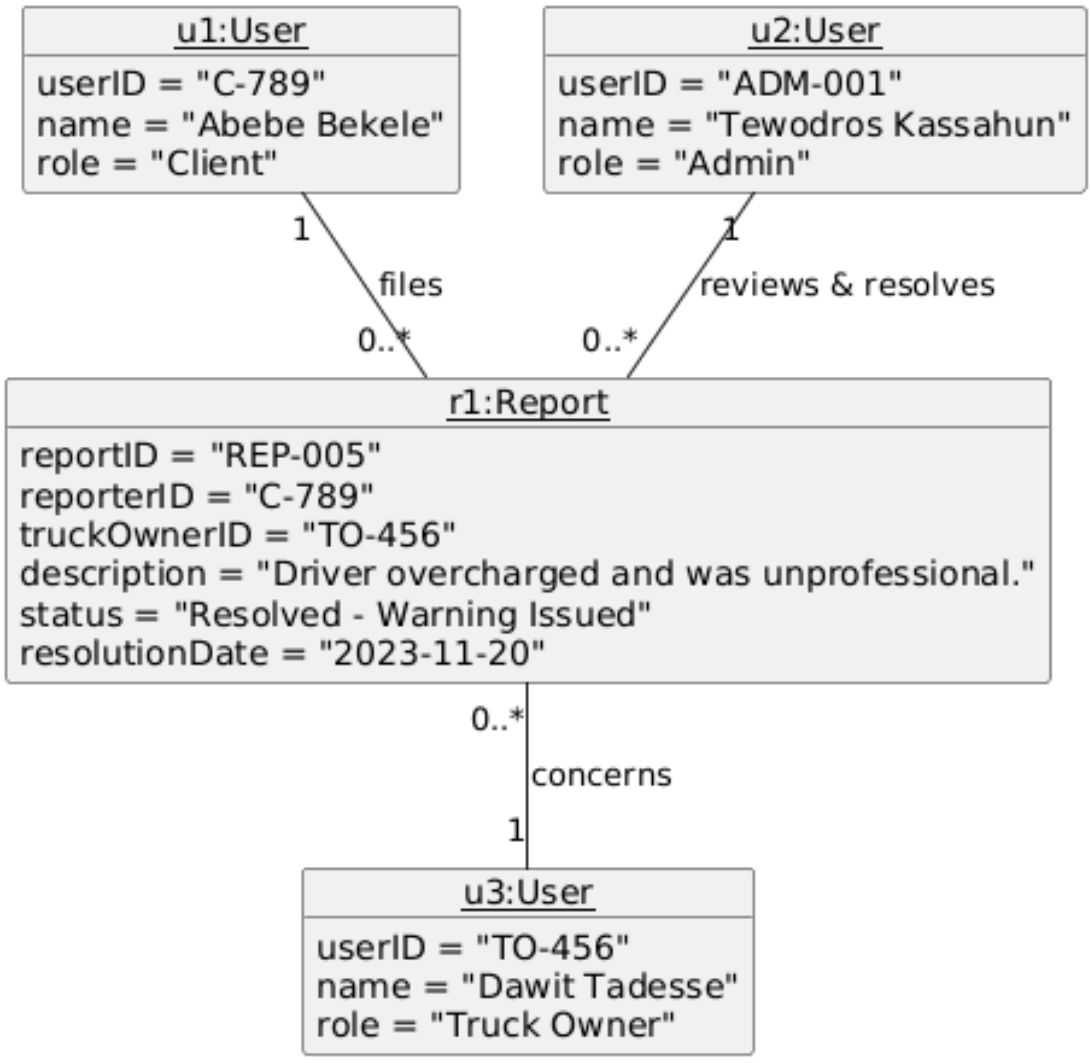


Figure 3.7: Accountability, Trust & Dispute Resolution Diagram

3.3.5. Access Control and Security

The Kuli system utilizes a Role-Based Access Control (RBAC) model to manage system resources and user privacy. Access privileges are categorized by the four primary user roles defined in the system: **Admin**, **Truck Owner**, **Client**, and **Call-Center Assistant**.

Table 3.1: Access Control List (ACL) Table

Packages	Subsystems	Operations	Admin	Client	Truck Owner	Assistant
Edge & Security	Identity Manager	login() / logout()	X	X	X	X
		verifyOTP()	X	X	X	X
		resetPassword()	X	X	X	X
Account Management	User Profile	updateProfile()	X	X	X	X
		suspendAccount()	X			
	Vehicle Registry	registerVehicle()			X	
		verifyVehicle()	X			
Logistics Engine	Booking Manager	createRequest()		X		X
		cancelRequest()		X		X
	Trip Controller	acceptRequest()			X	
		updateTripStatus()			X	
		sendMessage()		X	X	
Settlement & Quality	Finance Manager	confirmPayment()			X	

		viewTransactions()	X	X	X	
	Quality Control	submitRating()		X		X
		resolveDispute()	X			
System Utilities	Audit & Logs	viewAuditLogs()	X			
	Notifications	configureAlerts()	X	X	X	X

Security Issues and Mechanisms

- **Authentication Mechanism:** The system uses **Supabase Auth** to handle identity management. Users are authenticated via OTP/email or OAuth2 providers. Upon successful login, the system issues JSON Web Tokens (JWT) for session management, utilizing short-lived access tokens (15–60 minutes) and secure refresh tokens to maintain session integrity.
- **Encryption:** To ensure data confidentiality, the system employs **TLS (HTTPS)** for all data in transit between the client and backend. For data at rest, sensitive fields (such as National IDs) and database volumes are encrypted using industry-standard algorithms (e.g., AES-256). Passwords are never stored in plain text and are secured using salted hashes (bcrypt/Argon2).
- **Key Management:** Cryptographic keys, API secrets, and database credentials are managed via a dedicated cloud **Key Management Service (KMS)** or secrets manager (e.g., Vault). Access to these keys is restricted strictly by role, and keys are rotated periodically or upon suspicion of compromise to minimize security risks.

3.3.6. Subsystem Services

This section outlines the specific services provided by each logical subsystem within the Kuli

Backend Engine. Each service is designed to be autonomous yet collaborative, ensuring a robust and maintainable architecture.

- **Identity & Access (IAM):** Handles user authentication, session management, and Role-Based Access Control (RBAC) to ensure secure access for Clients, Truck Owners, Assistants, and Admins.
- **User Accounts (UA):** Manages the lifecycle of user profiles, including contact information, preferences, and the association of users with specific system roles.
- **Truck Registry (TR):** Orchestrates the supply-side data, managing vehicle enrollment, document storage for compliance, and the verification status of the fleet.
- **Booking & Pricing (BP):** Functions as the marketplace coordinator, validating logistics requests, calculating costs based on distance/class, and performing proximity-based truck matching.
- **Live Operations (LO):** Tracks the active state of logistics trips, managing status transitions (e.g., *Pending* to *In-Transit*) and maintaining the context for communication between participants.
- **Financial Settlement (PM):** Governs the fiscal records of the platform, including transaction logging, manual invoice generation for cash-based workflows, and audit trail maintenance.
- **Feedback & Disputes (FD):** Oversees platform quality by managing user ratings, text reviews, and the formal mediation process for reports handled by Administrators.
- **System Utilities (SU):** Provides cross-cutting infrastructure services, primarily focusing on the centralized notification dispatch hub (SMS/Email/Push) and system-wide audit logging.

3.4.Detailed Class Diagram

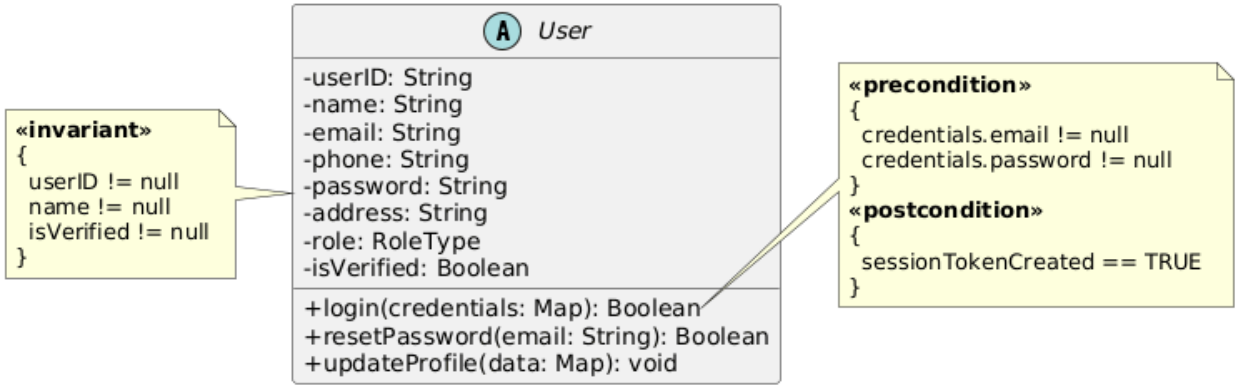


Figure 2.1.4.1. User (Base Class) detailed class diagram

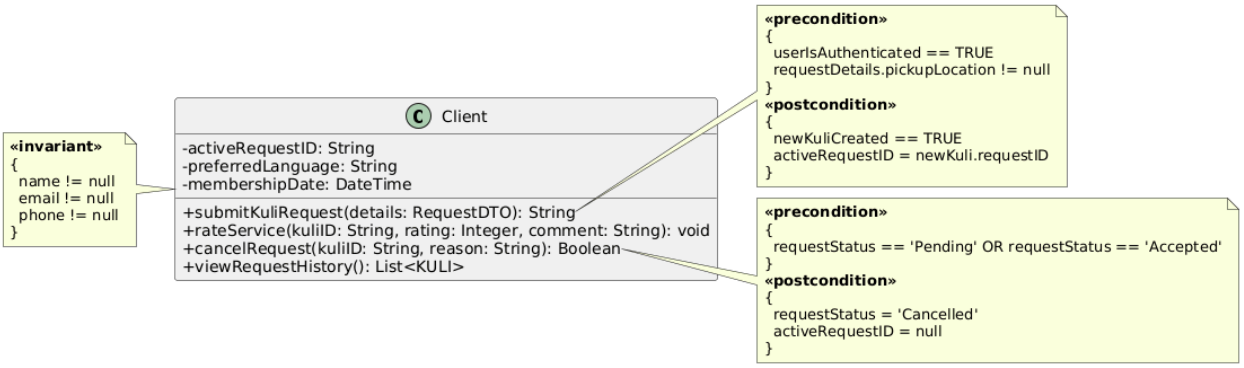


Figure 3.8: Client Entity detailed class diagram

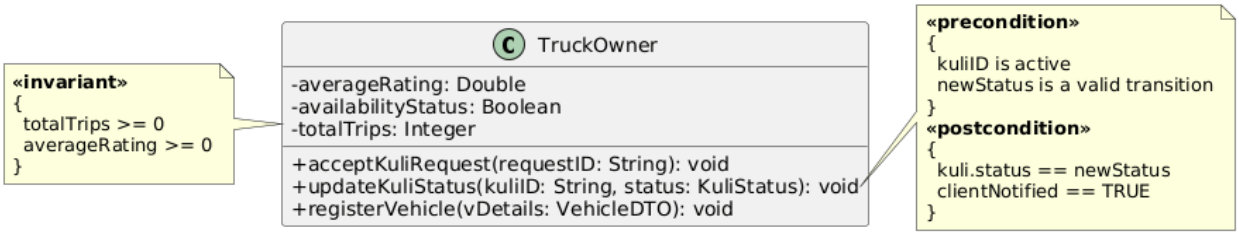


Figure 3.9 Truck Owner Entity

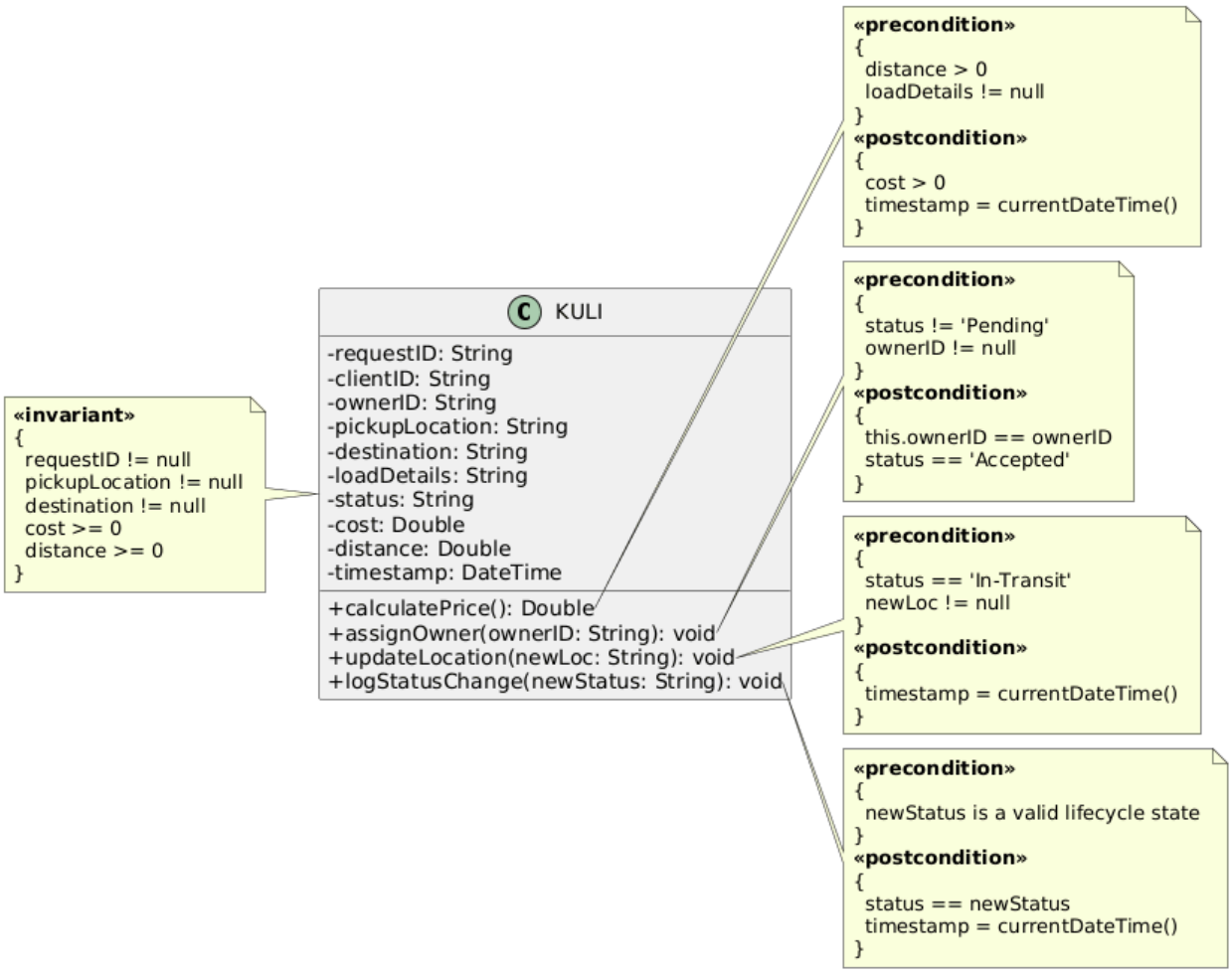


Figure 3.10: KULI (Logistics Request) Entity

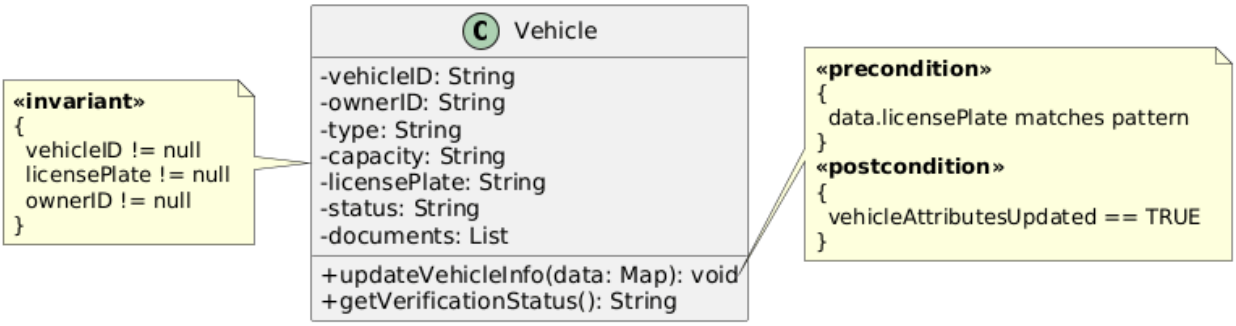


Figure 3.12: Vehicle Entity

3.5. Packages

This section describes the logical grouping of the Kuli subsystems into five domain-driven packages. This organization ensures that the system is modular, allowing for easier maintenance and the ability to scale specific domains independently as the platform grows.

Package Overview

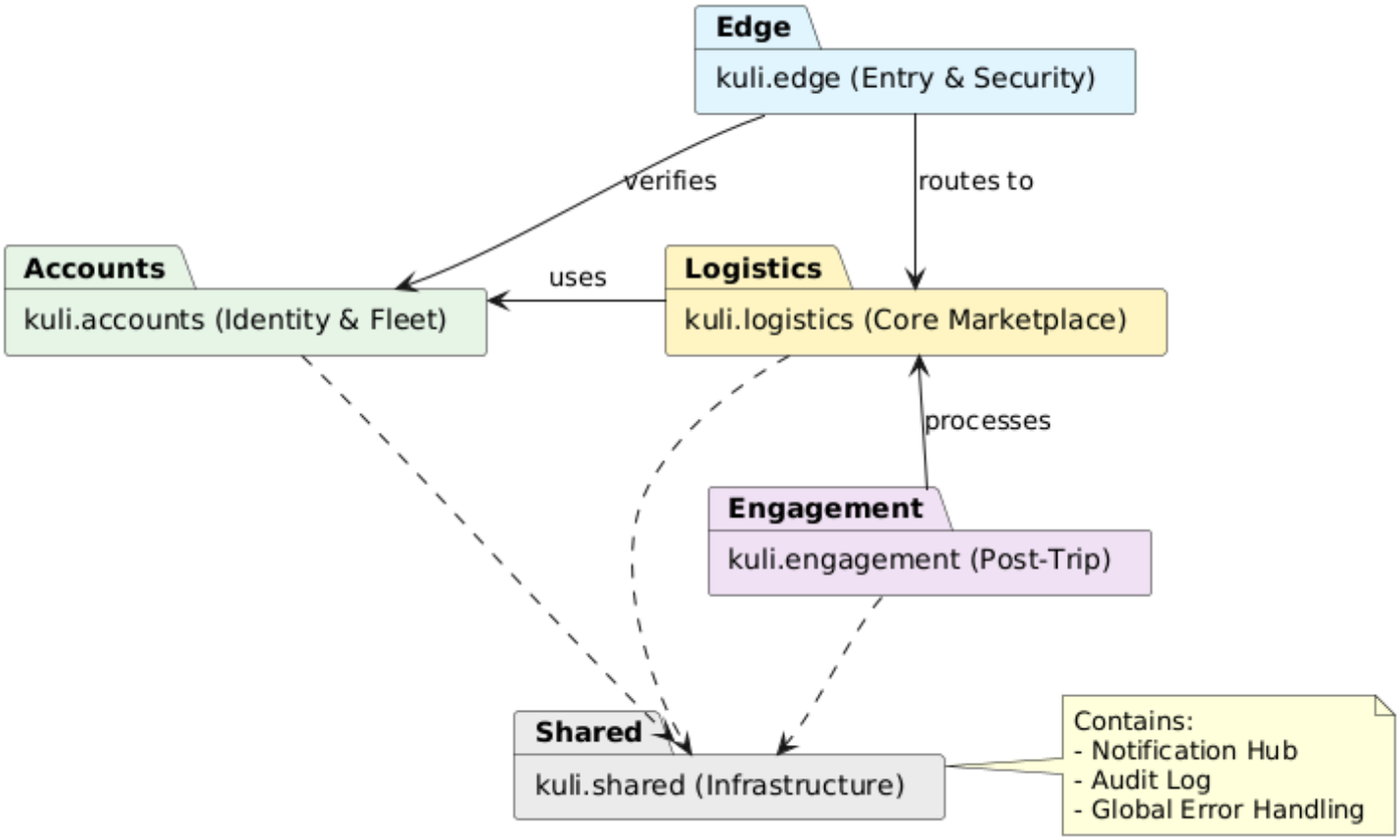
The following table maps the identified subsystems to their respective high-level packages.

Table 3.2: subsystem

Domain Package	Contained Subsystems	Primary Responsibility
kuli.edge	Access & Gateway (AG), Identity (ID)	Entry point security, routing, and user authentication.

kuli.accounts	User Accounts (UA), Truck Registry (TR)	Management of client profiles, truck owner data, and fleet verification.
kuli.logistics	Booking & Pricing (BP), Live Operations (LO)	Core business logic: request matching, pricing, and real-time trip execution.
kuli.engagement	Financial Settlement (PM), Feedback & Disputes (FD)	Post-trip processing: billing, payments, and quality assurance/reviews.
kuli.shared	System Utilities (SU), Common	Shared infrastructure: notifications, logging, and global utilities.

3.5.1 Package Diagram


Edge
kuli.edge (Entry & Security)
verifies
routes to
Accounts
kuli.accounts (Identity & Fleet)
uses
Logistics
kuli.logistics (Core Marketplace)
processes
Engagement
kuli.engagement (Post-Trip)
Shared
kuli.shared (Infrastructure)
Contains:
- Notification Hub
- Audit Log
- Global Error Handling

REFERENCES

1. Addis Ababa University, Department of Computer Science, Undergraduate Final Project Guideline.
2. https://reactnative.dev/docs/getting-started
3. https://www.geeksforgeeks.org/unified-modeling-language-uml-sequence-diagrams/
4. https://online.visual-paradigm.com/diagrams/tutorials/deployment-diagram-tutorial/
5. https://www.trade.gov/country-commercial-guides/ethiopia-transport-and-logistics
6. https://www.forbes.com/sites/forbestechcouncil/2021/08/17/the-uberization-of-logistics/
7. https://doi.org/10.1016/j.trpro.2020.08.001
8. https://www.wrike.com/agile-guide/agile-development-life-cycle/
9. https://swimm.io/learn/system-design/system-design-complete-guide-with-patterns-examples-and-techniques
10. https://en.wikipedia.org/wiki/Last_mile_(transportation